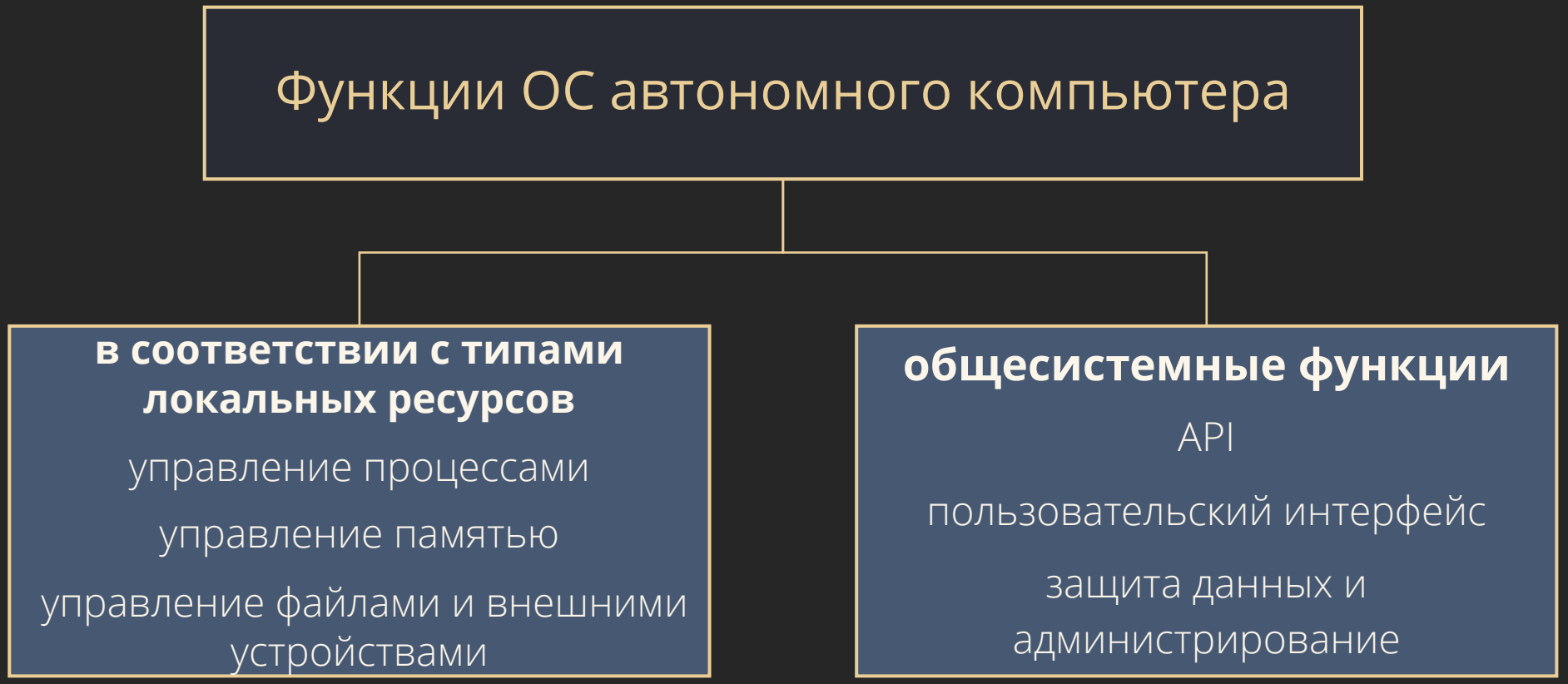
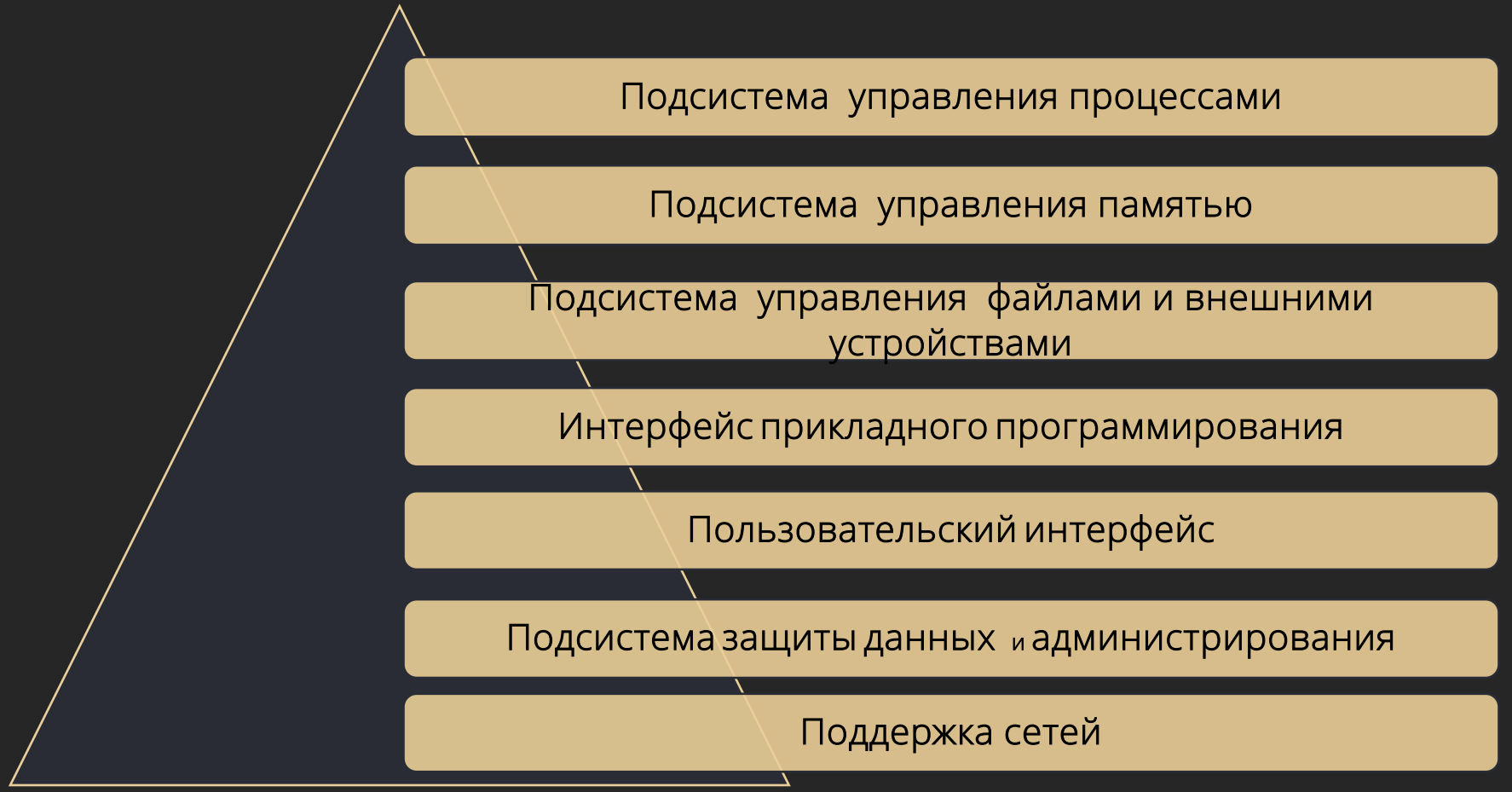
Функциональные компоненты ОС

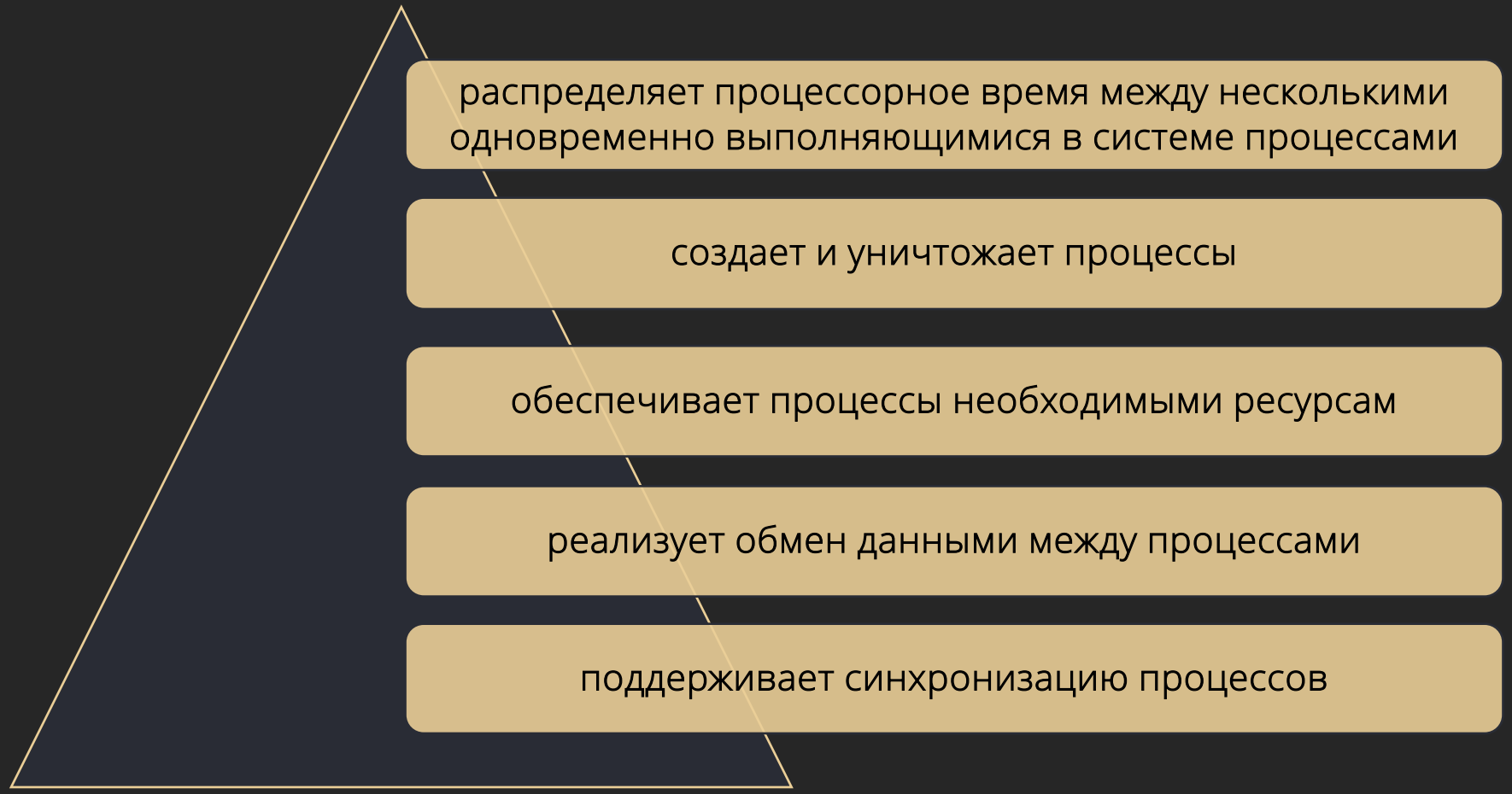
Функции операционных систем



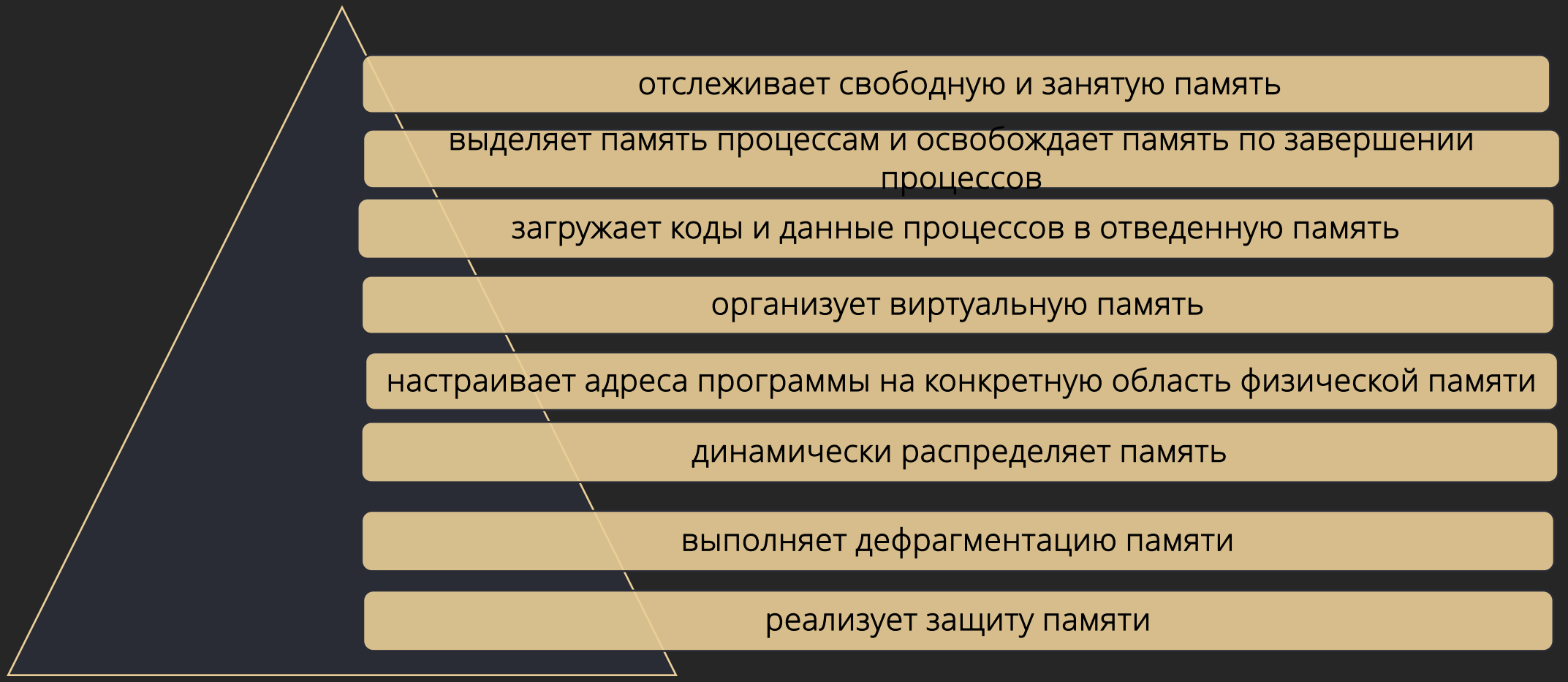
Функциональные компоненты операционной системы автономного компьютера



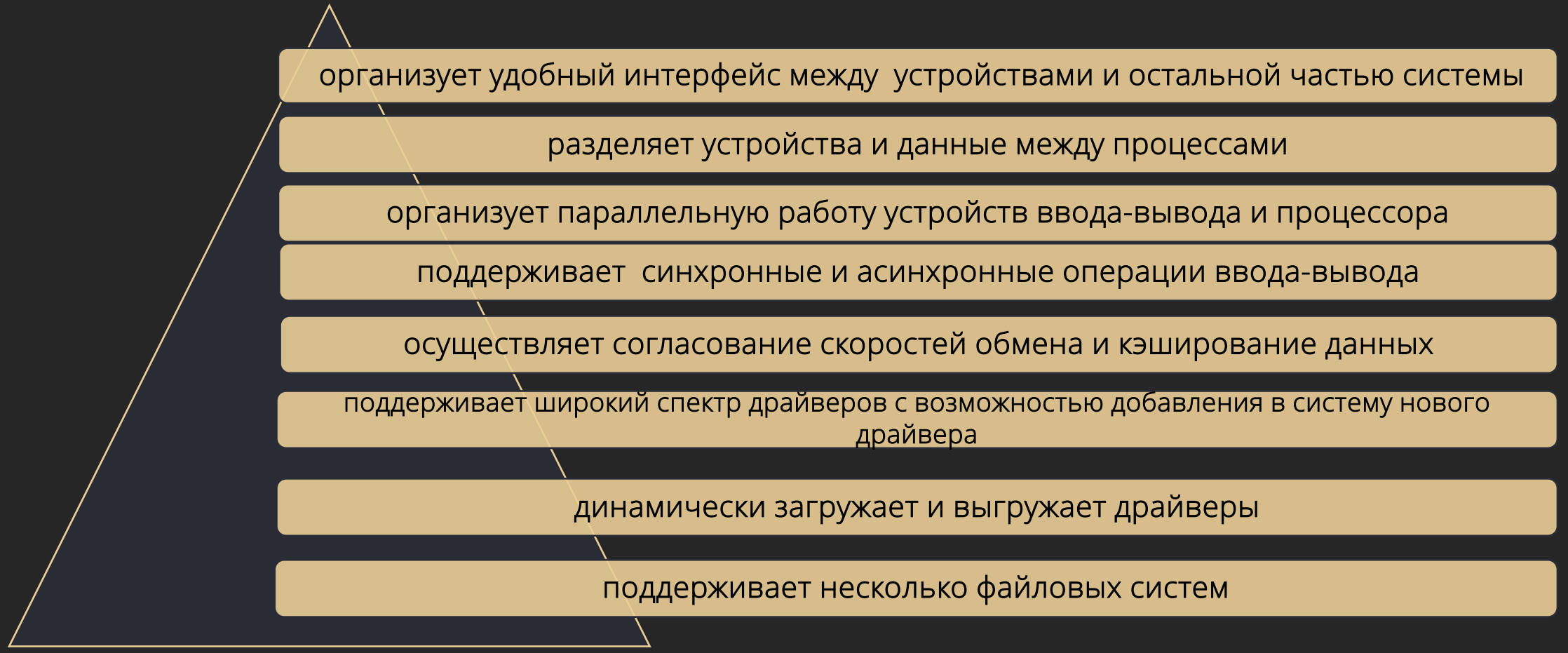
Подсистема управления процессами



Подсистема управления памятью



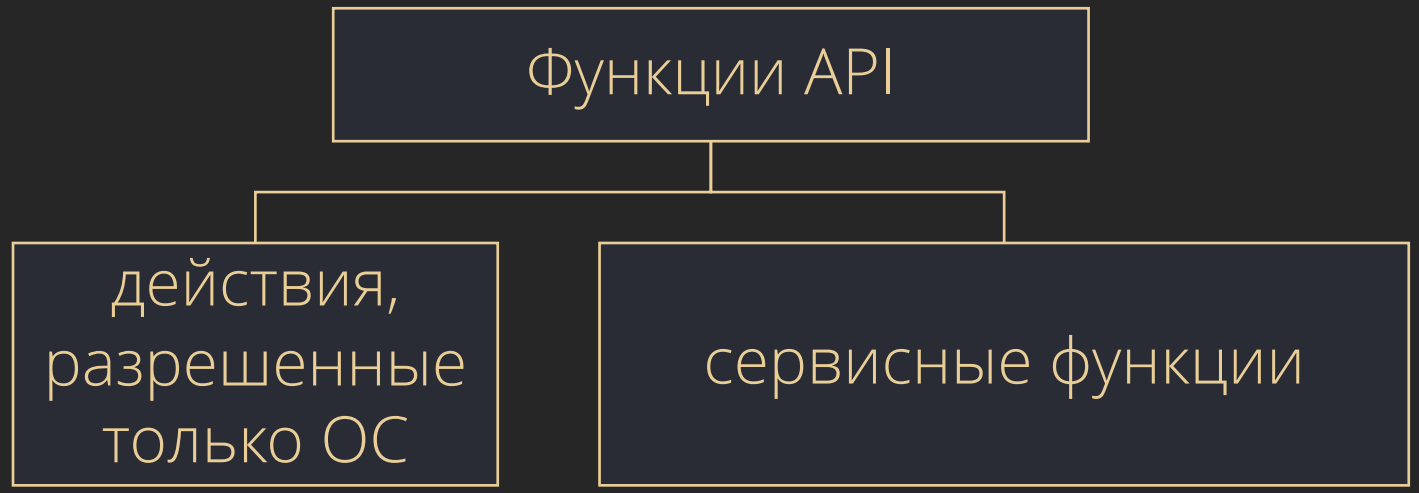
Подсистема управления файлами и внешними устройствами



Интерфейс прикладного программирования

Возможности ОС доступны прикладному программисту виде набора функций, которые составляют интерфейс прикладного программирования (Application Programming Interface, API).

- ✓ Приложения выполняют обращения к ОС с помощью **системных вызовов**.



Системный вызов (SysCall)

Системный вызов - это обращение из кода в User Mode (User Space) к коду ядра:





Системные вызовы реализуются с помощью **механизма программных прерываний**.

Системные вызовы

Реализация системных вызовов должна обеспечивать:

- переключение в привилегированный режим
- высокую скорость вызова процедур ОС
- по возможности единообразное обращение к системным вызовам для всех аппаратных платформ, для которых предназначена ОС
- возможность расширения набора системных вызовов
- контроль со стороны ОС использования системных вызовов



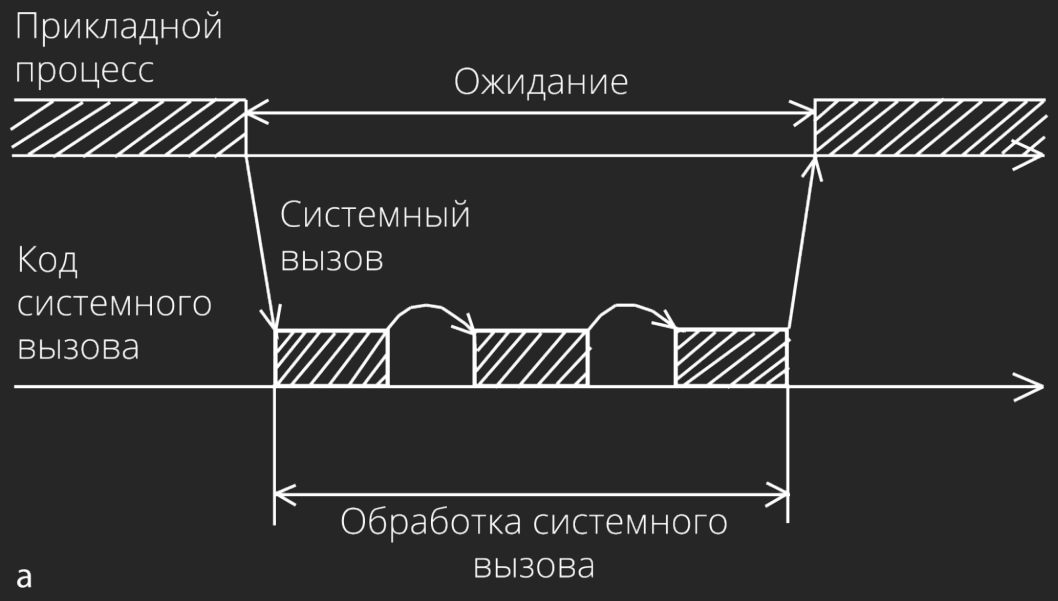
Системные вызовы реализуются с помощью механизма программных прерываний.

- ✓ Для разработчика приложений все особенности конкретной ОС представлены особенностями ее API. Поэтому разные ОС с одинаковым набором API, кажутся им одной и той же ОС. Это упрощает стандартизацию ОС. Стандартом API UNIX является стандарт **Posix**.

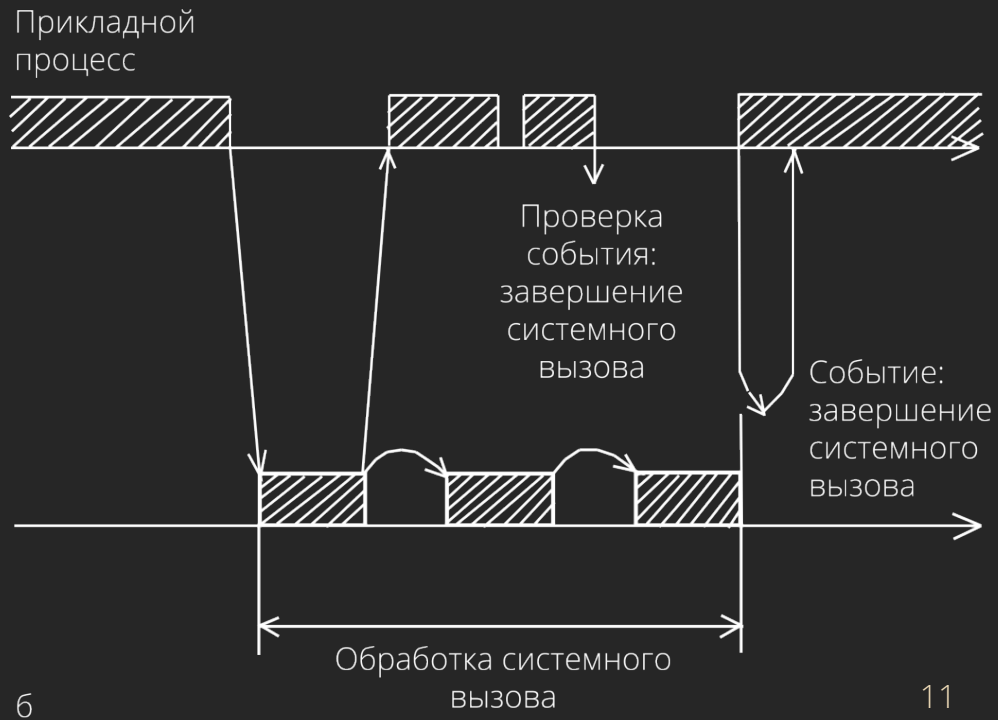


Системные вызовы

- синхронные
- асинхронные



а



б

Схемы обработки системных вызовов

- децентрализированная
- централизированная





Приблизительное соответствие функций API в Windows и *nix

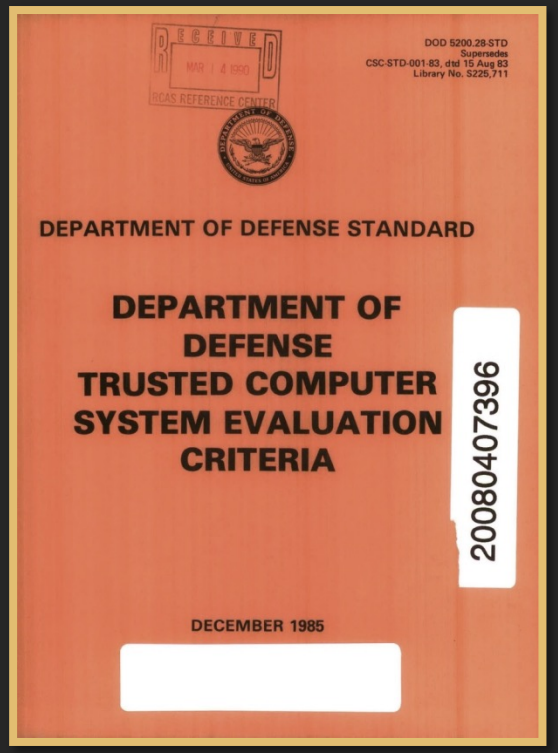
UNIX	Win32	Описание
fork	CreateProcess	Создает новый процесс
waitpid	WaitForSingleObject	Ожидает завершения процесса
execve	(нет)	CreateProcess=fork+execve
exit	ExitProcess	Завершает выполнение процесса
open	CreateFile	Создает файл или открывает существующий файл
close	CloseHandle	Закрывает файл
read	ReadFile	Читает данные из файла
write	WriteFile	Записывает данные в файл
lseek	SetFilePointer	Перемещает указатель файла
stat	GetFileAttrbutesEx	Получает различные атрибуты файла
mkdir	CreateDiectory	Создает новый каталог
rmdir	RemoveDirectory	Удаляет пустой каталог
link	(нет)	Win32 не поддерживает связи
unlink	DeleteFile	Удаляет существующий файл
mount	(нет)	Win32 не поддерживает подключение к файловой системе
umount	(нет)	Win32 не поддерживает подключение к файловой системе
chdir	SetCurrentDirectory	Изменяет рабочий каталог

Защита данных и администрирование

- защита от сбоев и отказов аппаратуры (реализуется путем резервирования) и ошибок программного обеспечения (например, самовосстанавливающиеся файловые системы)
- защита от несанкционированного доступа (процедура логического входа, аутентификация, авторизация, администрирование, аудит)



Критерии оценки безопасности компьютерных систем



Оранжевая книга" - стандарт МО США № 5200.28

Класс D: Minimal Protection (Минимальная защита) – к этому классу относятся системы, подобные DOS, так как защита в них практически отсутствует.

Класс C: Discretionary Protection (Избирательная защита) – большинство широко используемых операционных систем претендуют на соответствие защите по классу C2. Однако, есть разница между сертификацией по классу C2 в Национальном центре компьютерной безопасности (National Computer Security Center, NCSC) и просто соответствием уровню защищенности класса C2.

Класс C1: Discretionary Security Protection (Избирательная защита).

Класс C2: Controlled Access Protection (Управляемый доступ).

Класс B: Mandatory Security Protection (Обязательная защита) – система должна иметь математическое описание своей защиты и поддерживать защиту даже в отключенном состоянии.

Класс B1: Labeled Security Protection (Защита с применением меток безопасности)

Класс B2: Structured Protection (Структурированная защита)

Класс B3: Security Domains (Домены безопасности).

Класс A: Verified Protection (Верифицированная защита) – для проверки спецификаций такой системы применяются методы формальной верификации.

Класс A1: Verified Design (Верифицированная разработка)

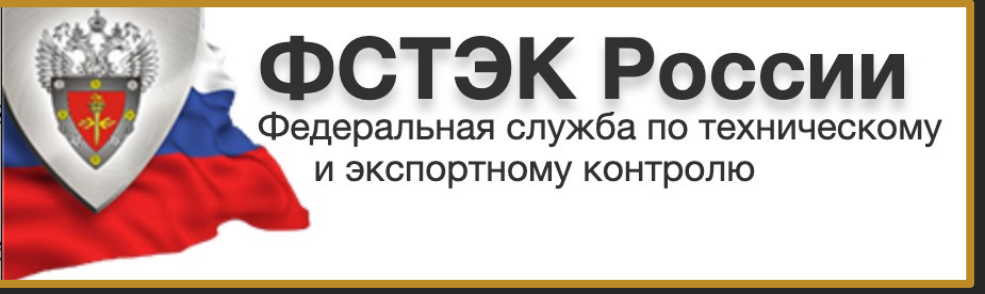
Требования для класса безопасности C2:



- Владелец ресурса (например, файла) должен иметь возможность контролировать доступ к нему.
- ОС должна защищать объекты от их повторного использования другими процессами.
- Пользователи должны идентифицировать себя при помощи уникальных имен и паролей, пригодных для отслеживания деятельности пользователей.
- Администраторы должны иметь возможность аудита связанных с безопасностью событий, журналы событий должны быть защищены от несанкционированного доступа.
- ОС должна защищать саму себя от изменений.

Common Criteria

- Текущий стандарт оценки безопасности, используемый в С США и во многих других странах, называется общими критериями (Common Criteria, CC).
- Стандарт CC включает понятие профиля защиты, используемое для сбора требований безопасности в свободно определяемые и сопоставляемые наборы, и понятие задания по безопасности —Security Target, содержащего набор требований по безопасности, которые могут быть выдвинуты на основе профиля защиты. Стандарт CC также определяет диапазон из семи оценочных уровней доверия, которые показывают уровень доверия к сертификации.



ИНФОРМАЦИОННОЕ СООБЩЕНИЕ ОБ УТВЕРЖДЕНИИ ТРЕБОВАНИЙ БЕЗОПАСНОСТИ ИНФОРМАЦИИ К ОПЕРАЦИОННЫМ СИСТЕМАМ от 18 октября 2016 г. N 240/24/4893

В соответствии с Требованиями выделяются следующие типы операционных систем:

- операционная система общего назначения (тип "А") – операционная система, предназначенная для функционирования на средствах вычислительной техники общего назначения (автоматизированные рабочие места, серверы, смартфоны, планшеты, телефоны и иные);
- встраиваемая операционная система (тип "Б") – операционная система, встроенная (прошитая) в специализированные технические устройства, предназначенные для решения заранее определенного набора задач;
- операционная система реального времени (тип "В") – операционная система, предназначенная для обеспечения реагирования на события в рамках заданных временных ограничений при заданном уровне функциональности.

Для дифференциации требований к функциям безопасности операционных систем выделяются шесть классов защиты операционных систем. Самый низкий класс – шестой, самый высокий – первый.

- Операционные системы, соответствующие 6 классу защиты, применяются в государственных информационных системах 3 и 4 классов защищенности* в автоматизированных системах управления производственными и технологическими процессами 3 класса защищенности** в информационных системах персональных данных при необходимости обеспечения 3 и 4 уровней защищенности персональных данных***.
- Операционные системы, соответствующие 5 классу защиты, применяются в государственных информационных системах 2 класса защищенности*, в автоматизированных системах управления производственными и технологическими процессами 2 класса защищенности**, в информационных системах персональных данных при необходимости обеспечения 2 уровня защищенности персональных данных***.
- Операционные системы, соответствующие 4 классу защиты, применяются в государственных информационных системах 1 класса защищенности*, в автоматизированных системах управления производственными и технологическими процессами 1 класса защищенности**, в информационных системах персональных данных при необходимости обеспечения 1 уровня защищенности персональных данных***, в информационных системах общего пользования II класса****.
- Операционные системы, соответствующие 1, 2 и 3 классам защиты, применяются в информационных (автоматизированных) системах, в которых обрабатывается информация, содержащая сведения, составляющие государственную тайну.

https://fstec.ru/normotvorcheskaya/informatsionnye-i-analiticheskie-materialy/1206-informatsionnoe-soobshchenie-fstek-rossii-ot-18-oktyabrya-2016-g-n-240-24-4893

Пользовательский интерфейс



алфавитно-цифровой (командный) интерфейс



графический (WIMP: Window, Image, Menu, Pointer) интерфейс



голосовой (SILK: Speech, Image, Language, Knowledge) интерфейс



биометрическая технология

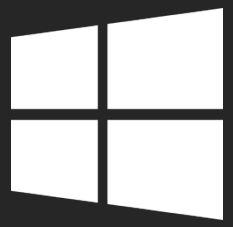
АРХИТЕКТУРЫ ОПЕРАЦИОННЫХ СИСТЕМ

Требования, предъявляемые к современным ОС

Главное требование - выполнение основных функций эффективного управления ресурсами и обеспечение удобного интерфейса для пользователя и прикладных программ.

1. прозрачность (незаметность) работы служебных программ
2. гарантированная надежность
3. максимальная скорость выполнения
4. минимальный машинный код
5. использование стандартных средств для связи с проблемными программами

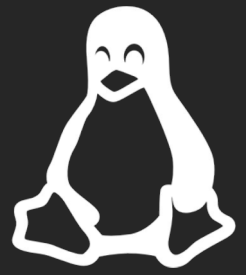
Сравнение количества строк кода в ОС Linux и Windows



Год	Версия	Строк кода
1993	Windows NT 3.1	5 600 000
1995	Windows NT 3.5	8 400 000
1996	Windows NT 4.0	16 000 000
2000	Windows 2000	29 000 000
2001	Windows XP	35 000 000
2007	Windows Vista	45 000 000
2009	Windows 7	42 000 000
2012	Windows 8	50 000 000
2015	Windows 10	55 000 000



Android 12 млн. строк

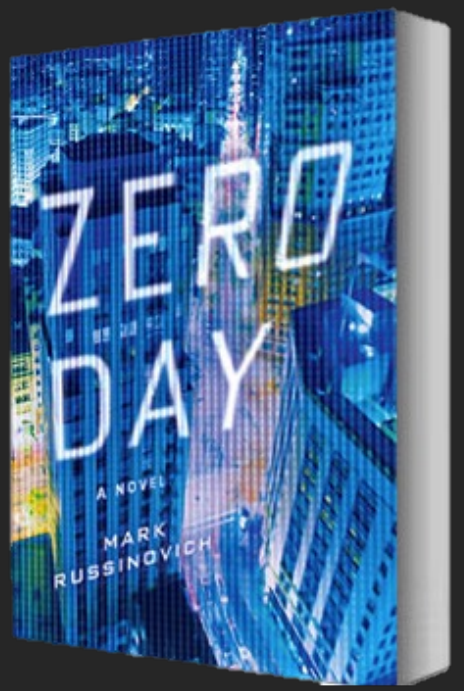


Год	Версия	Строк кода
1991	Ядро Linux 0.1	10 000
1994	Ядро Linux 1.0.0	176 000
1995	Ядро Linux 1.2.0	311 000
1996	Ядро Linux 2.0.0	778 000
1999	Ядро Linux 2.2.0	1 801 000
2001	Ядро Linux 2.4.0	3 338 000
2003	Ядро Linux 2.6.0	6 000 000
2009	Ядро Linux 2.6.32	12 607 000
2017	Ядро Linux 4.11.7	18 500 000
2017	Ядро Linux 4.12	24 100 000
2018	Ядро Linux 4.18	25 300 000
2019	Ядро Linux 5.2	26 550 000
2020	Ядро Linux 5.8	28 360 000

Дистрибутивы Linux «тяжелее». Больше всего строк в Debian, который имеет около 419 миллионов строк кода (почти в 4 раза больше, чем у Red Hat).

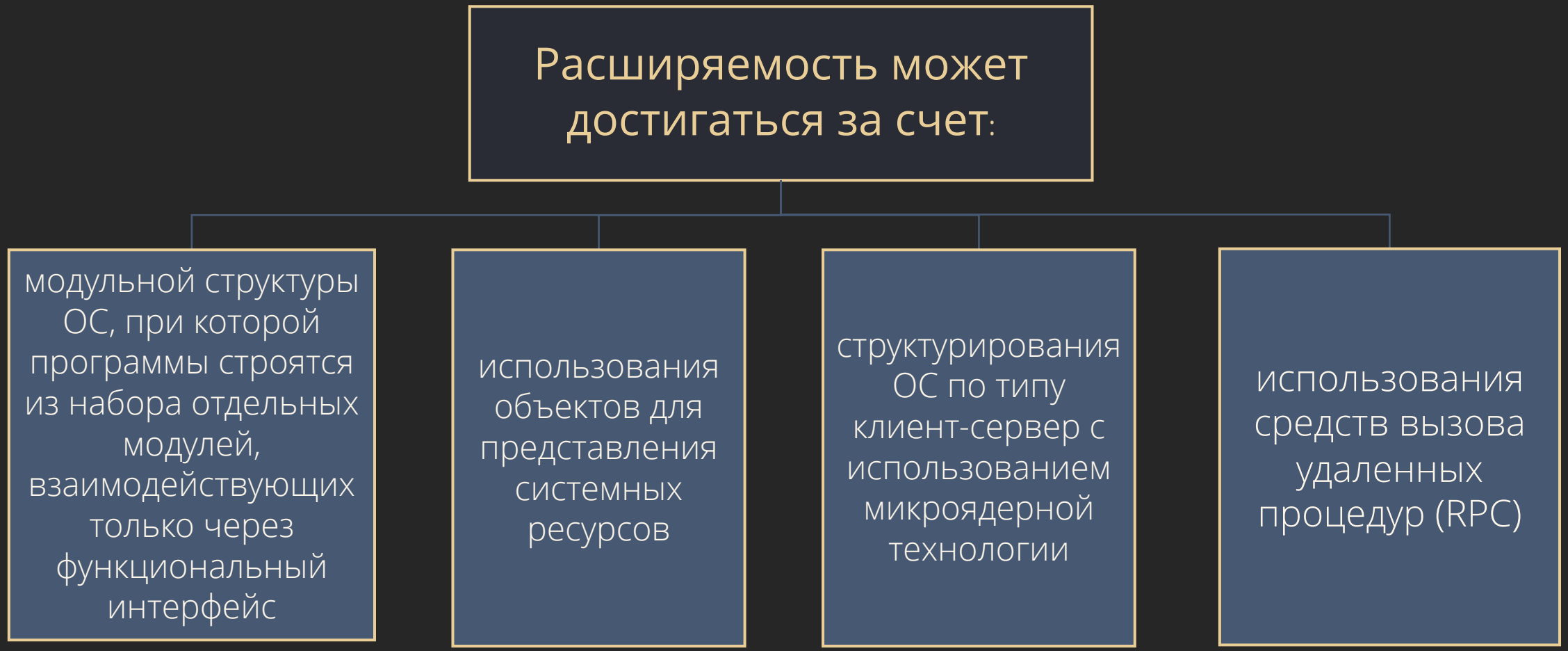
Уязвимость нулевого дня

Zero-day эксплойт — киберугроза, использующая ошибку или уязвимость в приложении или операционной системе и появившаяся сразу после обнаружения данной уязвимости, пока разработчики ПО еще не успели создать патч, а IT-администраторы — принять другие меры безопасности. Происхождение термина связано с тем, что уязвимость или атака становится публично известна до момента выпуска производителем ПО исправлений ошибки (то есть потенциально уязвимость может эксплуатироваться на работающих копиях приложения без возможности защититься от неё).



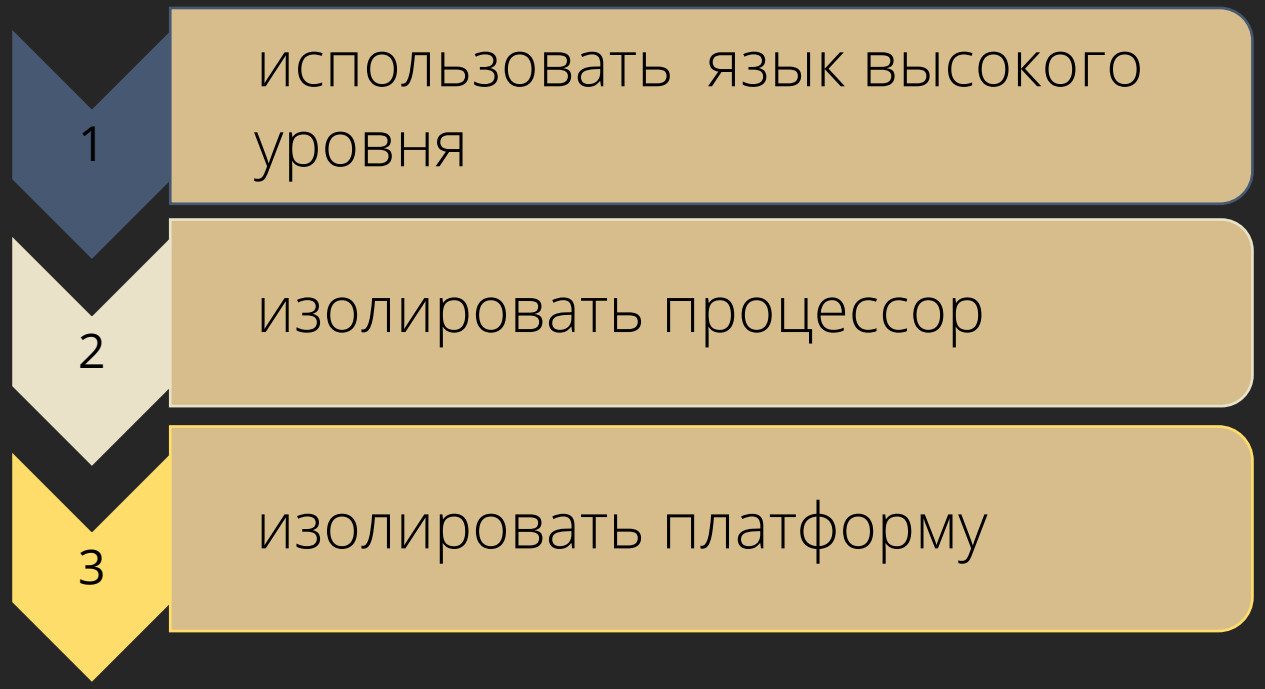
Расширяемость

Код должен быть написан так, чтобы систему можно было легко наращивать и модифицировать по мере изменения потребностей рынка.



Переносимость

Переносимость (многоплатформенность) дает возможность перемещать всю систему на машину, базирующуюся на другом процессоре или аппаратной платформе, делая при этом по возможности минимальные изменения в коде.



Совместимость



Совместимость - способность ОС выполнять программы, написанные для других ОС или для более ранних версий данной операционной системы, а также для другой аппаратной платформы.

Совместимость

на уровне исходных кодов

требует наличия соответствующего компилятора в составе программного обеспечения, при этом необходима перекомпиляция имеющихся исходных текстов в новый исполняемый модуль, а также совместимость на уровне библиотек и системных вызовов)

двоичная

достигается в том случае, когда исполняемую программу можно запустить на выполнение в вычислительной системе с другой ОС, для этого необходимы: совместимость на уровне команд процессора

При совпадении архитектур процессоров (набора команд и диапазона адресов) двоичная совместимость достигается при:

1. поддержке вызовов API-функций новой ОС
2. соответствии внутренней структуры исполняемого файла правилам новой ОС

Для достижения двоичной совместимости в случае различных архитектур, кроме этих мер, необходимы :

1. эмуляция
2. использование множественных прикладных программных сред
3. система виртуальных машин (СВМ)

Система виртуальных машин (СВМ)



Вариант организации вычислительного процесса, при котором на одном компьютере одновременно выполняются несколько копий одной и той же или нескольких разных ОС. Каждая из этих ОС функционирует так же, как если бы она выполнялась на отдельном компьютере.



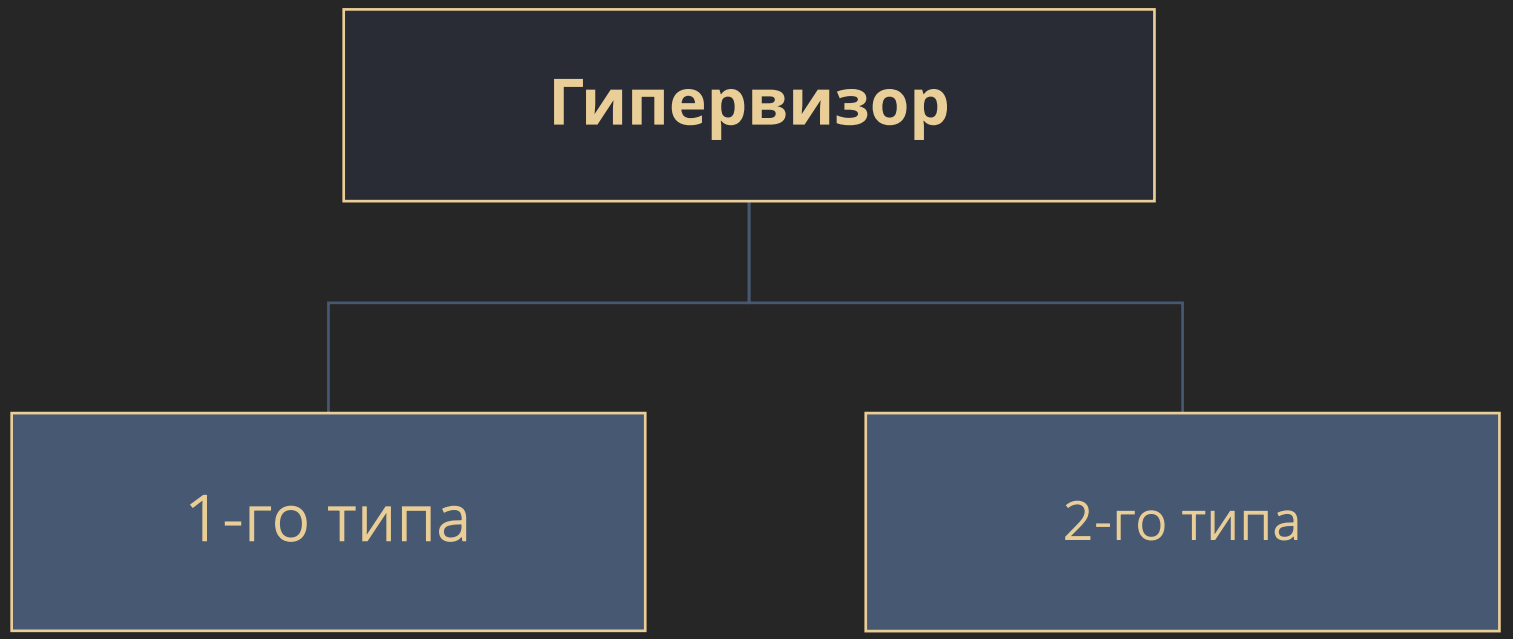
Виртуальная машина — программная и/или аппаратная система, эмулирующая аппаратное обеспечение некоторой платформы* и исполняющая программы для нее на host-платформе ** или виртуализирующая некоторую платформу и создающая на ней среды, изолирующие друг от друга программы и даже операционные системы.



Помимо процессора, ВМ может эмулировать работу как отдельных компонентов аппаратного обеспечения, так и целого реального компьютера (включая BIOS, оперативную память, жёсткий диск и другие периферийные устройства).

*target — целевая, или гостевая платформа
** host — хост-платформа, платформа-хозяин

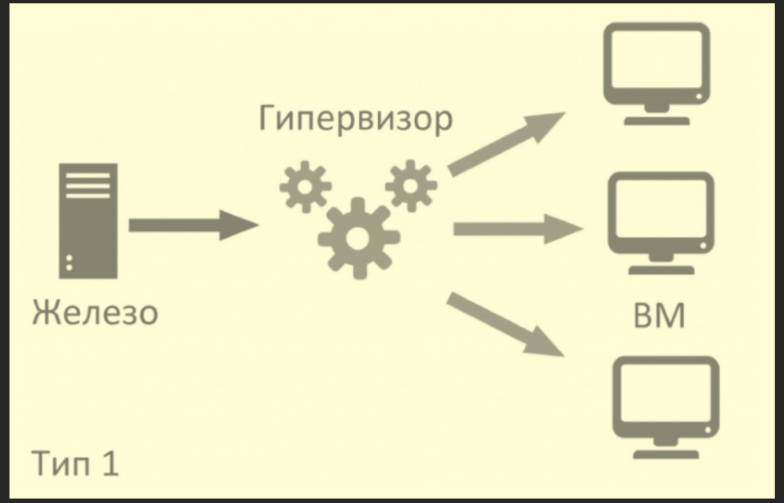
Основная идея заключается в том, что **монитор виртуальных машин** (Virtual Machine Monitor (**VMM**)) создает иллюзию присутствия нескольких (виртуальных) машин на одном и том же физическом оборудовании. VMM известен также как **гипервизор.**

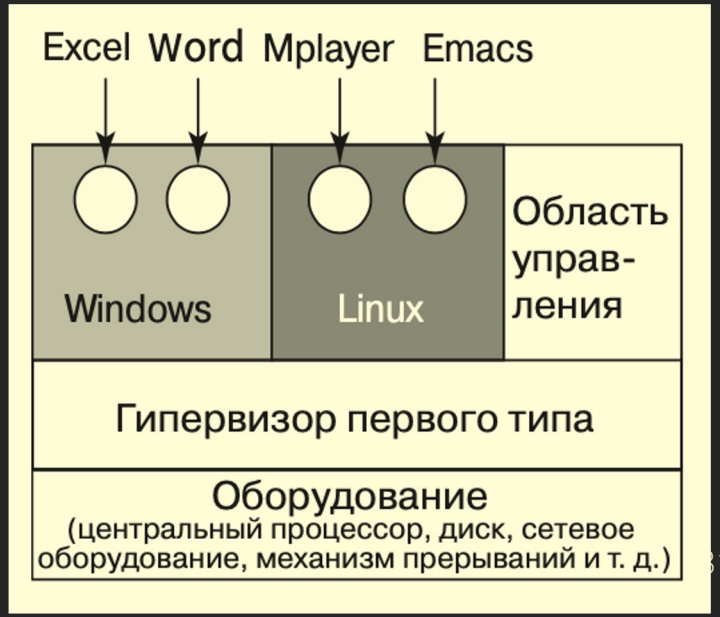


Гипервизор первого типа

иногда называются автономными гипервизорами). Запускаются на аппаратном обеспечении хоста, управляют оборудованием и виртуальными машинами. Этот гипервизор похож на ОС, поскольку это единственная программа, запущенная в самом привилегированном режиме.

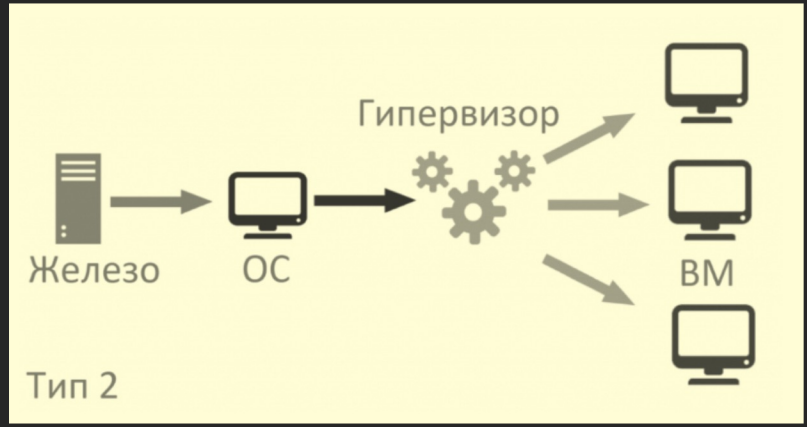
К этому типу относятся гипервизоры: VMware ESX / ESXi, Microsoft Hyper-V, Xen, Oracle VM Server для SPARC, и Oracle VM Server для архитектуры x86.



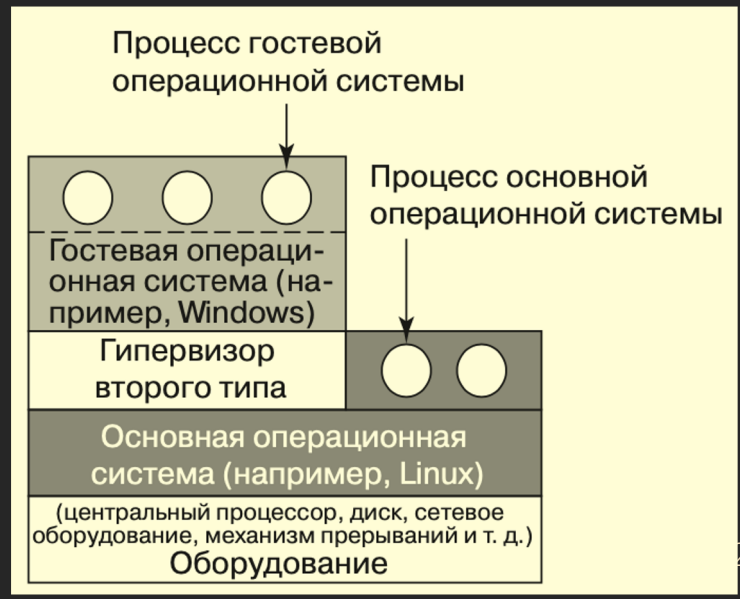


Гипервизоры второго типа

иногда называются хостовыми гипервизорами. Запускаются на ОС как любые другие процессы. В этом случае гипервизор разделяют ОС машины хоста и ОС гостевой машины, а гостевая ОС представлена обычным процессом хоста

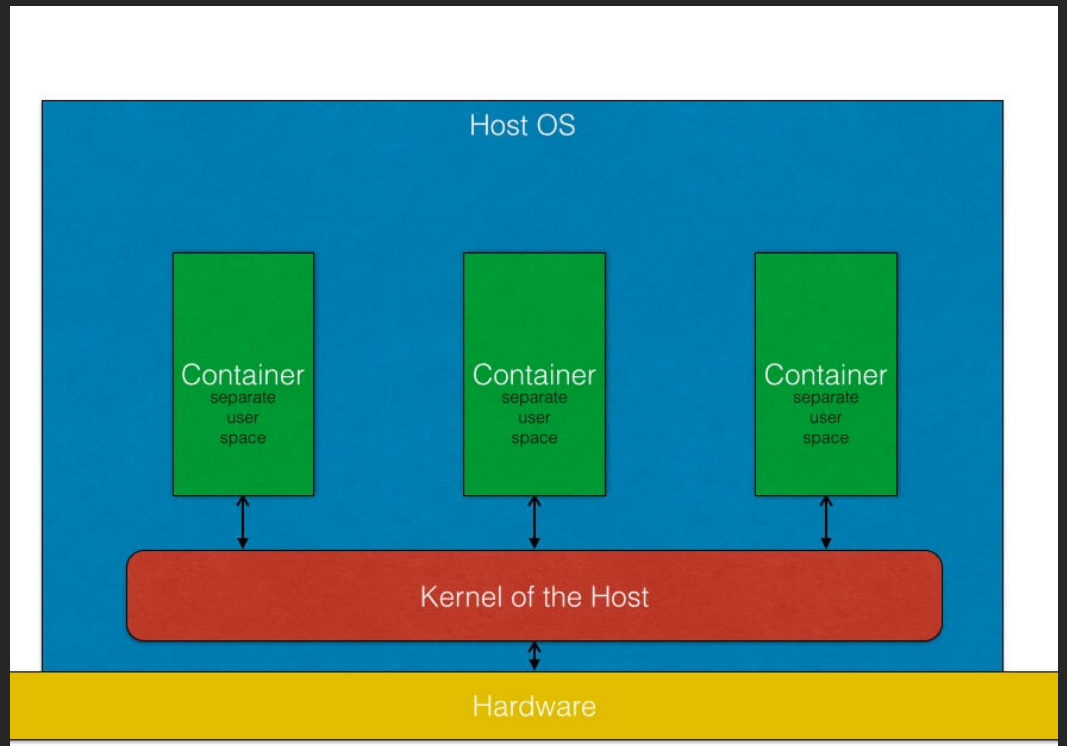


К этому типу гипервизоров относятся: VMware Workstation, VMware Player, VirtualBox и Parallels Desktop для Mac.



Контейнеры

Контейнеризация – это легковесная виртуализация и изоляция ресурсов на уровне операционной системы, которая позволяет запускать приложение и необходимый ему минимум системных библиотек в полностью стандартизованном контейнере, соединяющемся с хостом или чем-либо внешним по отношению к нему при помощи определенных интерфейсов. Контейнер не зависит от ресурсов или архитектуры хоста, на котором он работает.
Все компоненты, необходимые для запуска приложения, упаковываются как один образ и могут быть использованы повторно. Приложение в контейнере работает в изолированной среде и не использует память, процессор или диск хостовой операционной системы. Это гарантирует изолированность процессов внутри контейнера



Operating System/Container Virtualization

Скорость создания. Контейнер можно создать быстрее, чем ВМ.

Экономичность. Контейнер занимает меньше места в хранилище, что уменьшает накладные расходы.

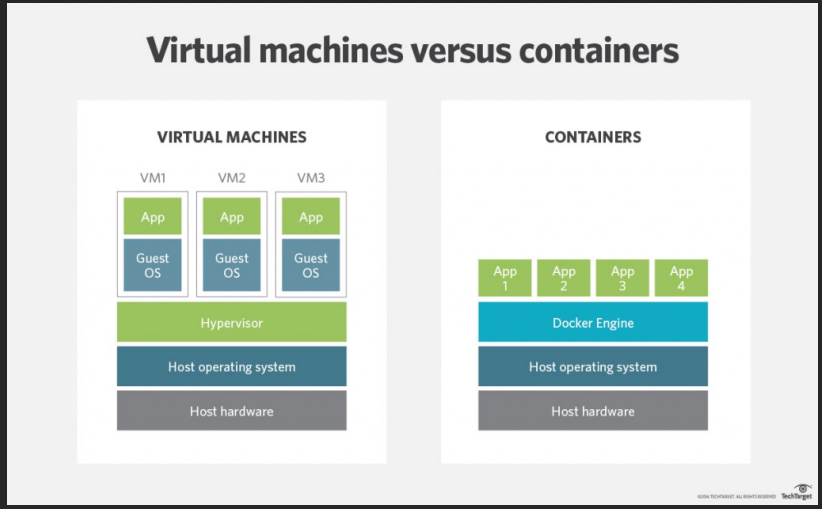
Высокая производительность. Отсутствие межсетевых зависимостей и конфликтов повышает производительность разработки. Каждый контейнер фактически представляет собой микросервис, который можно независимо обновлять, не задаваясь вопросом синхронизации.

Управление версиями. Можно мониторить версионность контейнеров, следить за различиями между ними.

Возможность миграции среды вычислений.

Стандартизация. Как правило, контейнеры создаются на основе открытых стандартов. Поэтому с ними можно работать в большинстве дистрибутивов Linux, Microsoft, MacOS. Все зависимости приложений и ОС, необходимые для работы приложения, инкапсулируются. Это позволяет без труда переносить образ контейнера из одной среды в другую.

Безопасность. Контейнеры изолированы друг от друга и базовой инфраструктуры. Изменение/обновление/удаление одного контейнера не влияет на другой.

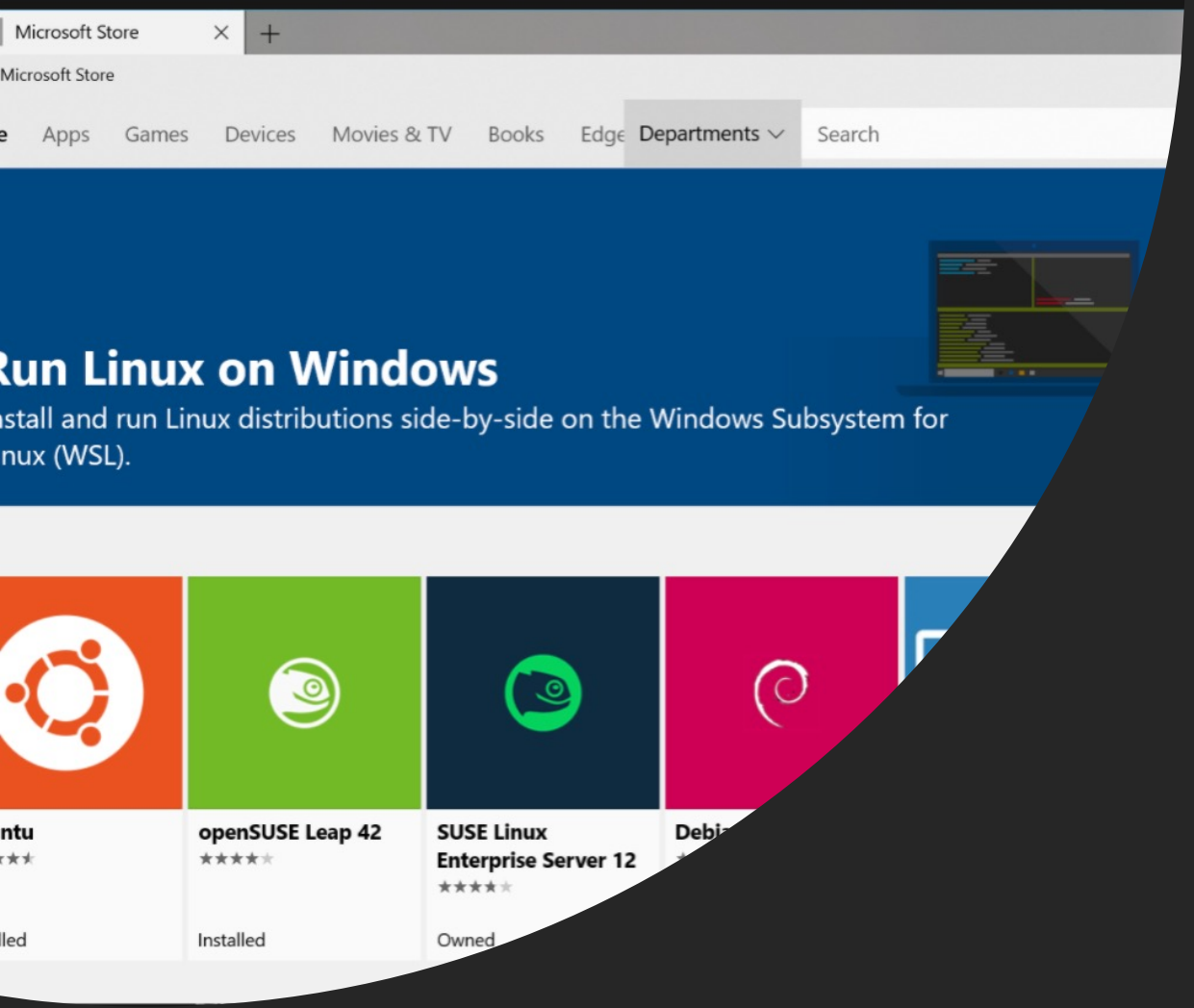


Высокая сложность. Рост количества контейнеров, работающих с приложением, влияет на сложность управления ими. В производственной среде для работы с множеством контейнеров стоит использовать оркестраторы. Например, Kubernetes и Mesos.

Разрастание. Нередко в контейнеры упаковывается гораздо больше ресурсов, чем реально требуется. Из-за этого образ разрастается, занимая больше места на диске.

Поддержка Native Linux. Docker и многие другие контейнерные технологии основаны на Linux-контейнерах (LXC). Из-за этого запуск контейнеров в Windows-среде не всегда удобен, а ежедневное использование сложнее, чем при работе в Linux.

Недостаточная зрелость. Технологии контейнеризации приложений появились на рынке сравнительно недавно. Не всегда удаётся сразу решить возникшую проблемы. Иногда требуется время на поиск решения.



WSL

Подсистема Windows для Linux позволяет разработчикам запускать среду GNU/Linux с большинством программ командной строки, служебных программ и приложений непосредственно в Windows без каких-либо изменений и необходимости использовать традиционную виртуальную машину или двойную загрузку.

https://docs.microsoft.com/ru-ru/windows/wsl/install-win10

https://m.habr.com/ru/company/vdsina/blog/535214/

WSL1

Подсистема Linux в ядре NT — это реализация уровня системных вызовов Linux перед ядром NT.

Эти системные вызовы позже делегируются примитивам NT, а не вызовам Win32. Самое главное: нет никакого перевода системных вызовов Linux в системные вызовы Win32.

Проблемы WSL1:

- WSL 1 должен реализовать все двоичные интерфейсы приложений (ABI) ядра Linux «бит к биту».

Если в интерфейсе есть ошибка, WSL 1 должен её воспроизвести.
Если есть функция, которая отсутствует в ядре NT, то она либо не может быть реализована, либо нуждается в дополнительной логике ядра (и поэтому становится медленнее).

- Подсистема Linux в WSL 1 должна соблюдать любые «ограничения» и внутренние различия, существующие между ядром NT и традиционным дизайном Unix.

Наиболее очевидным отличием является файловая система NTFS и её семантика, а также то, как эти различия вредят производительности бинарных файлов Linux.

WSL2

Образ ядра Linux в Windows 10 представляет собой легкую виртуальную машину.

Виртуальная машина внутри себя запускает обычное ядро Linux, файловую систему Linux и стандартный сетевой стек Linux. Всё это работает внутри VM.

Сравнение возможностей

Функция	WSL 1	WSL 2
Интеграция Windows и Linux	✅	✅
Быстрый запуск	✅	✅
Незначительный расход ресурсов	✅	✅
Запуск с использованием текущих версий VMware и VirtualBox	✅	✅
Управляемая виртуальная машина	❌	✅
Полнофункциональное ядро Linux	❌	✅
Полная совместимость системных вызовов	❌	✅
Производительность в файловых системах ОС	✅	❌

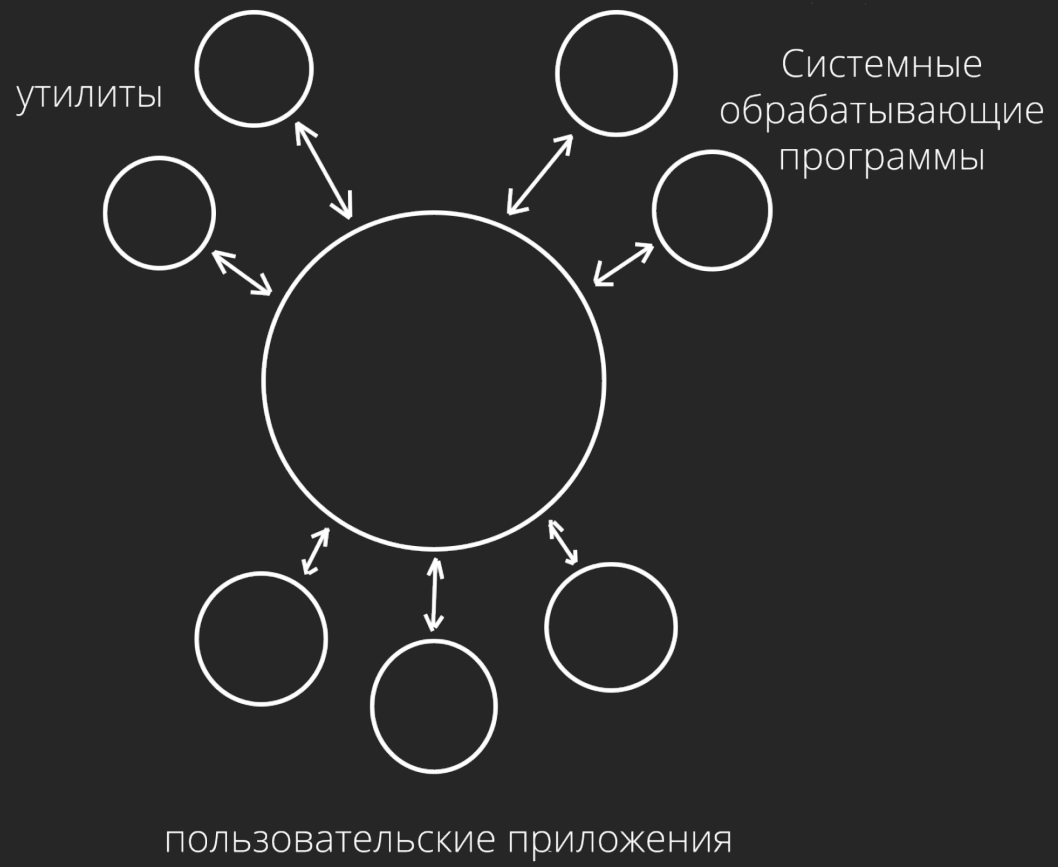
Wine

- Wine («Wine Is Not an Emulator»)— это программный инструмент для запуска игр и приложений, созданных для платформы Windows, в операционных системах на ядре Linux. В отличие от традиционных эмуляторов, он выполняет преобразование API-вызовов Windows в системные вызовы ядра Linux без использования виртуализации.

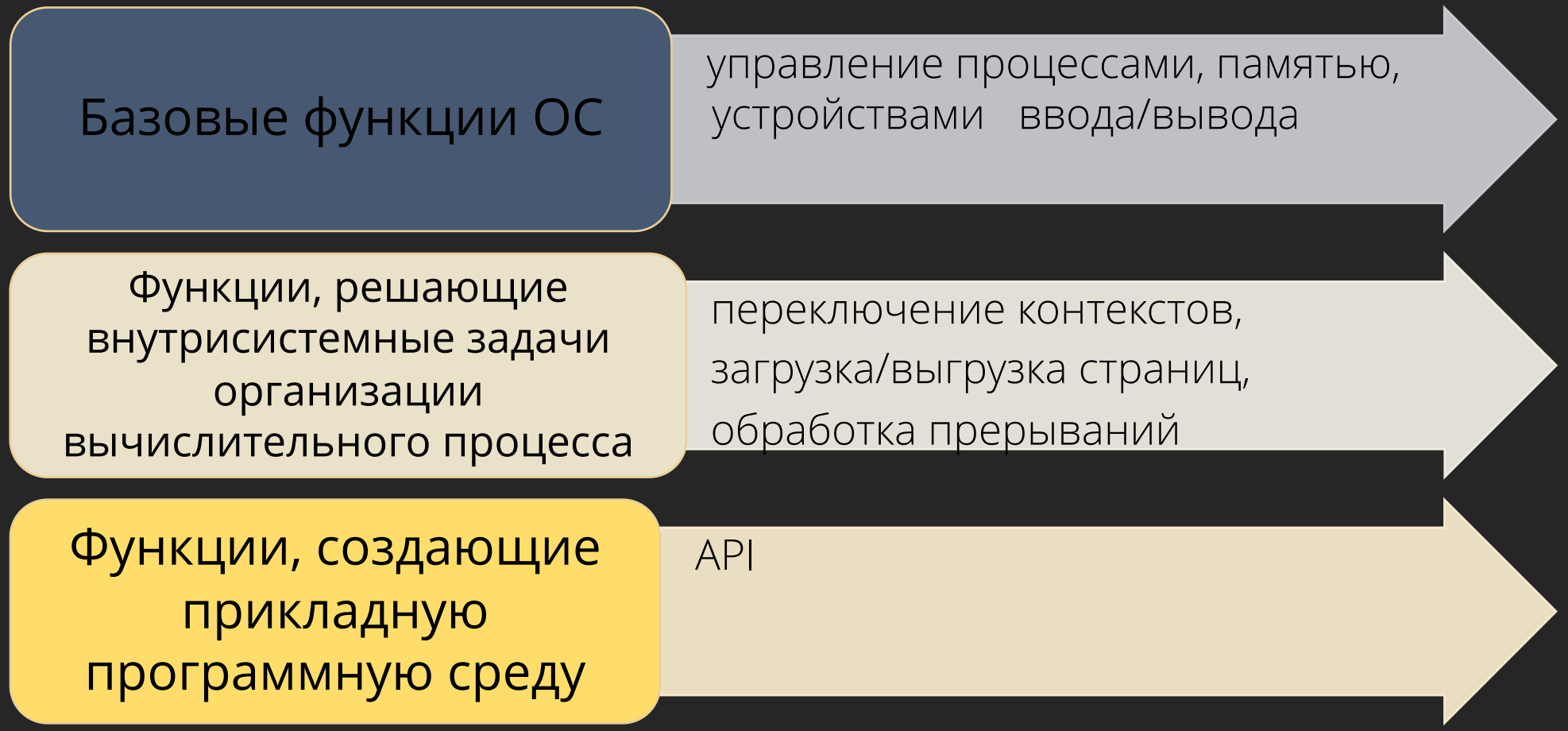
АРХИТЕКТУРА ОПЕРАЦИОННЫХ СИСТЕМ

Архитектура ОС





Ядро выполняет:



Модули ядра Linux

- Process Scheduler (SCHED) — планировщик процессов, отвечает за контроль над доступом процессов к CPU
- Memory Manager (MM) — менеджер памяти, обеспечивает различным процессам безопасный доступ к основной памяти системы
- Virtual File System (VFS) — виртуальная файловая система, создает абстрактный слой, скрывая детали оборудования, предоставляя общий файловый интерфейс для всех устройств
- Network Interface (NET) — сетевые интерфейсы, обеспечивает работу с различными сетевыми стандартами и сетевым оборудованием
- Inter-Process Communication (IPC) — межпроцессная подсистема, поддерживающая несколько механизмов для process-to-process связей в единой Linux-системе

Вспомогательные модули:

Утилиты

программы, решающие отдельные задачи управления и сопровождения компьютерной системы (дисковые утилиты: дефрагментаторы, очистка диска, разметка диска, программы сжатия, резервное копирование; утилиты работы с реестром; утилиты мониторинга оборудования и т.д.)

Системные обрабатывающие программы

текстовые и графические редакторы, компиляторы, компоновщики, отладчики

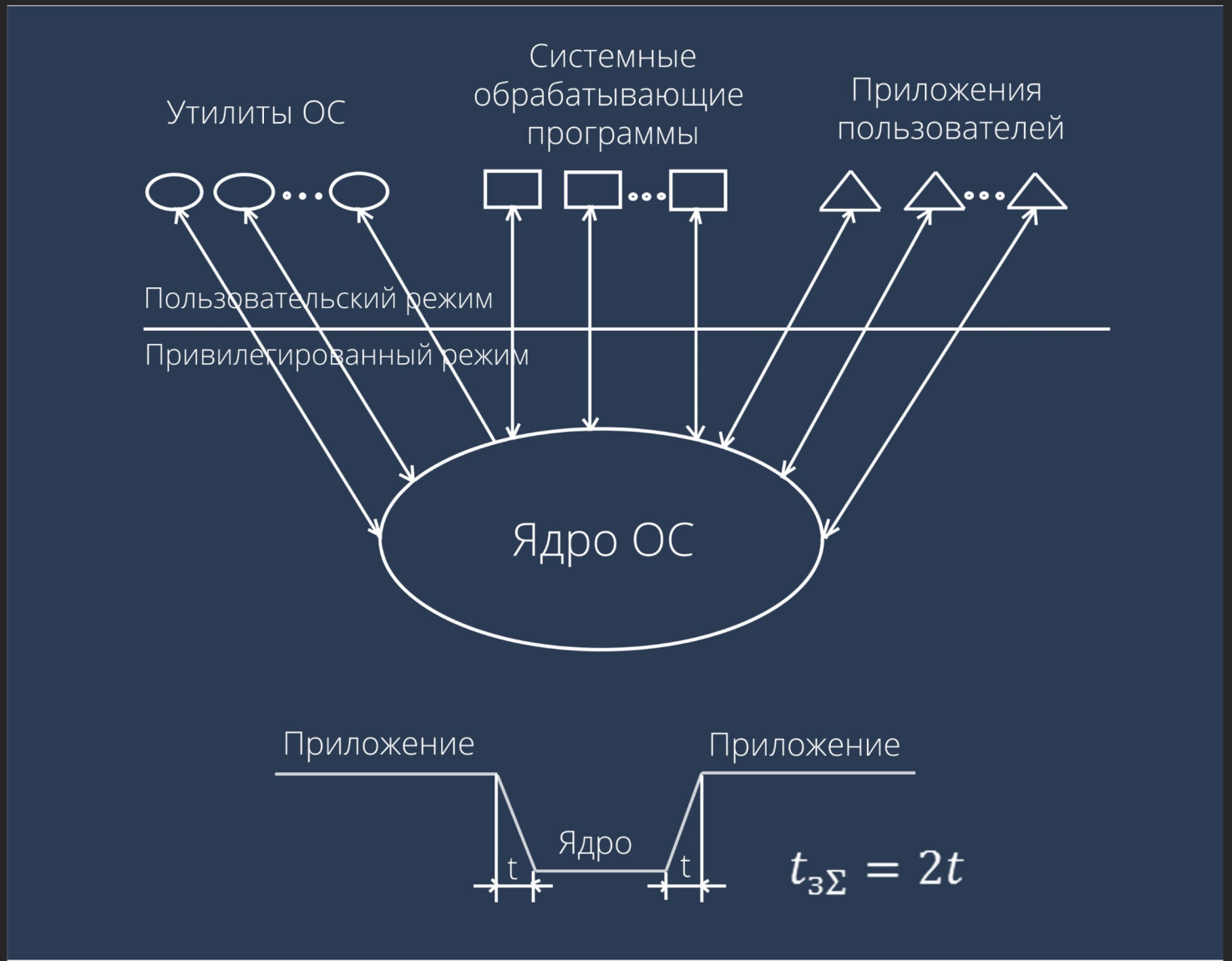
Программы дополнительных услуг

специальный вариант пользовательского интерфейса, калькулятор, игры

Библиотеки процедур

библиотеки математических функций, функций ввода/вывода и т.д.

Классическая архитектура

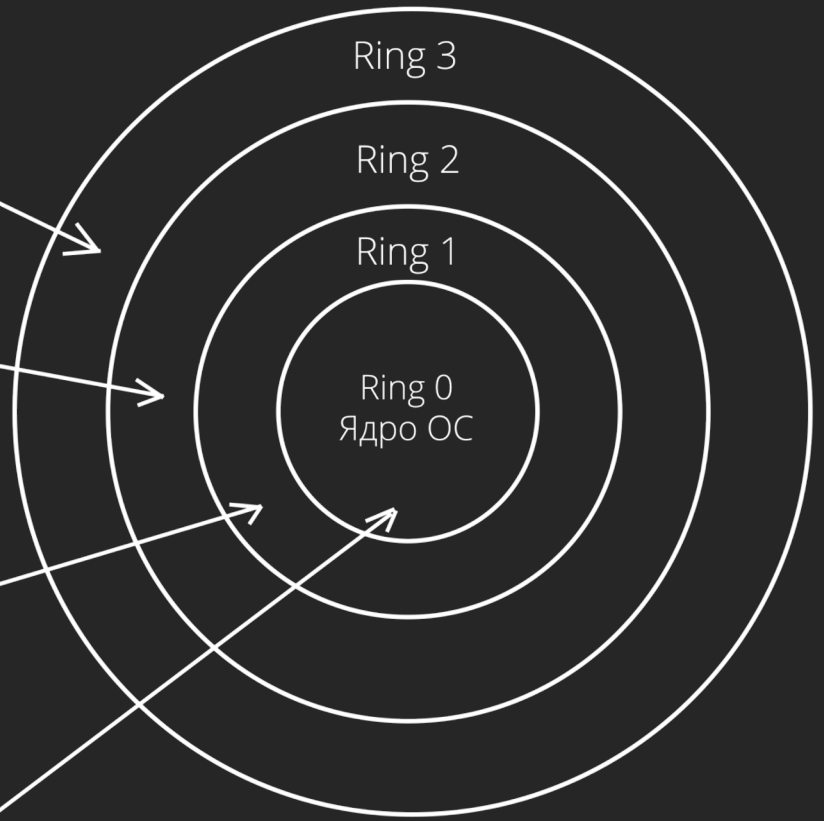


Пользовательский уровень - наименее привилегированный. Здесь работают прикладные программы

Уровень выполнения чуть более привилегированного кода, например программ с доступом к операциям ввода/вывода (использовался в OS/2)

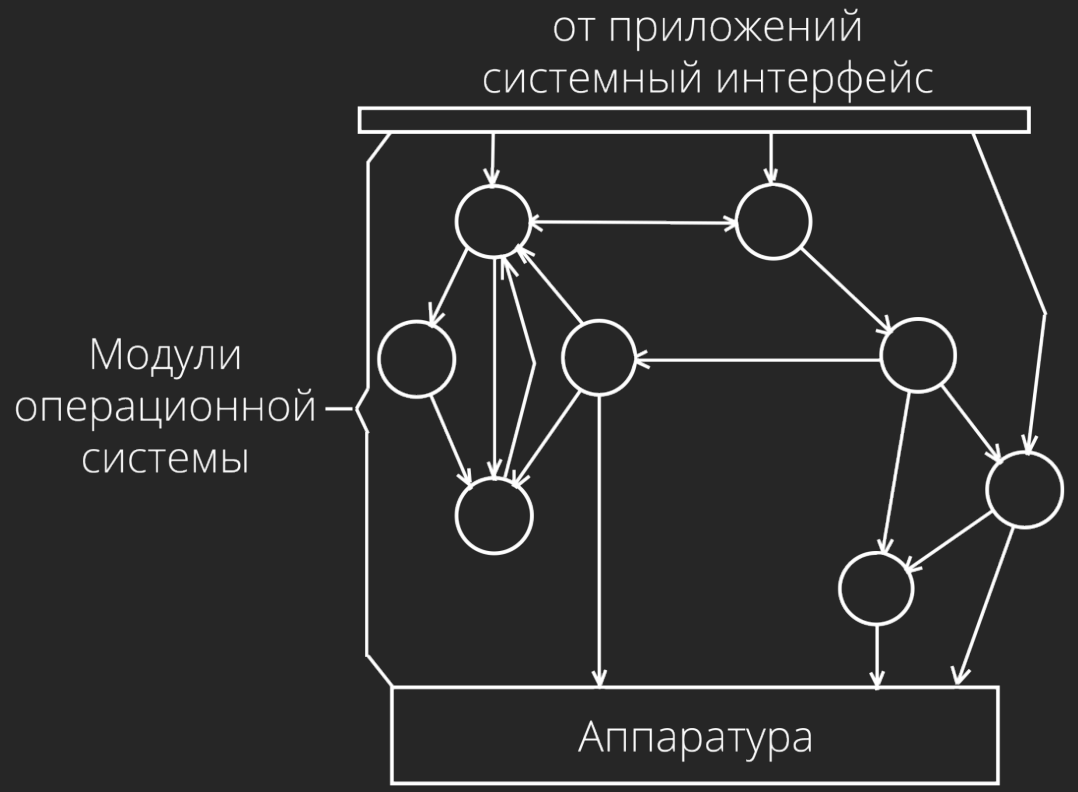
Еще более привилегированный уровень, может быть использован для некоторых драйверов, на практике редко задействован

Режим супервизора - самый привилегированный уровень. Здесь работает ядро ОС



Монолитная структура ОС

Вся ОС работает как единая программа в режиме ядра. Монолитная ОС написана как набор процедур, связанных в одну большую исполняемую программу. Каждая процедура имеет возможность при необходимости вызвать другую.

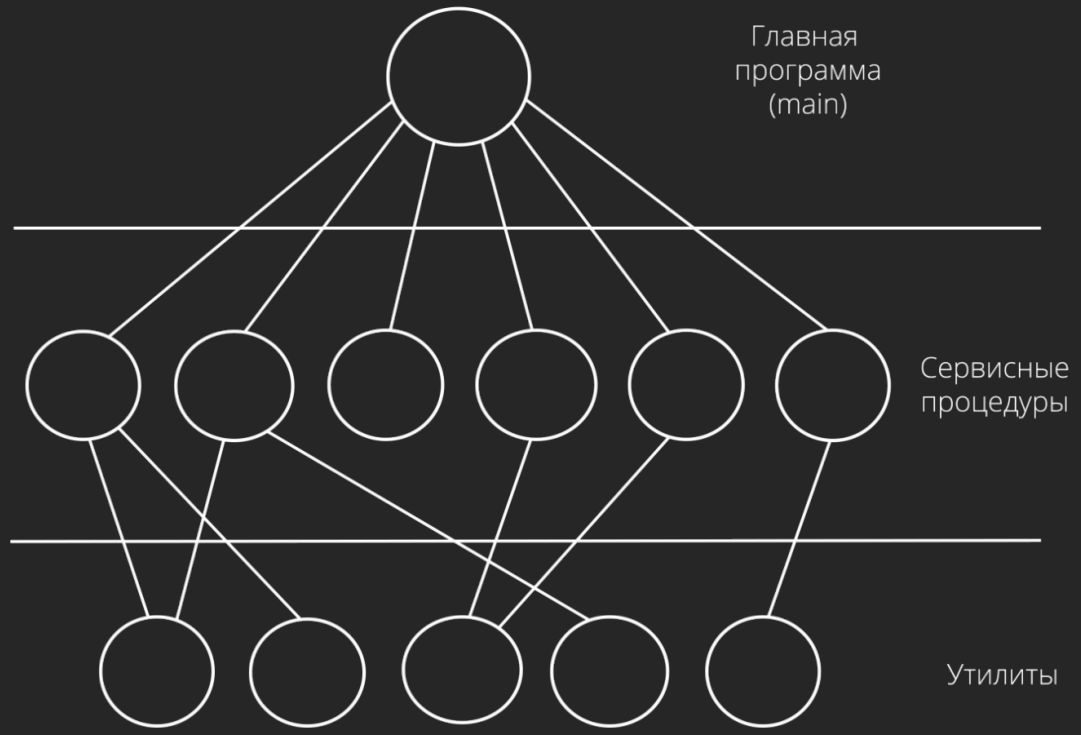


Монолитные системы могут быть структурированными:

Главная программа, которая вызывает требуемые сервисные процедуры.

Набор сервисных процедур, реализующих системные вызовы.

Набор утилит, обслуживающих сервисные процедуры.



Многоуровневые системы

Обобщением предыдущего подхода является организация ОС как иерархии уровней с хорошо определенными связями между ними, так чтобы объекты уровня N могли вызывать только объекты из уровня N-1.

Нижним уровнем в таких системах обычно является аппаратура, верхним уровнем интерфейс пользователя. Прикладные программы или модули самой операционной системы передают запросы вверх и вниз по этим уровням.

Плюсы:

- Многоуровневые системы хорошо реализуются.
- Слоеные системы хорошо модифицируются.

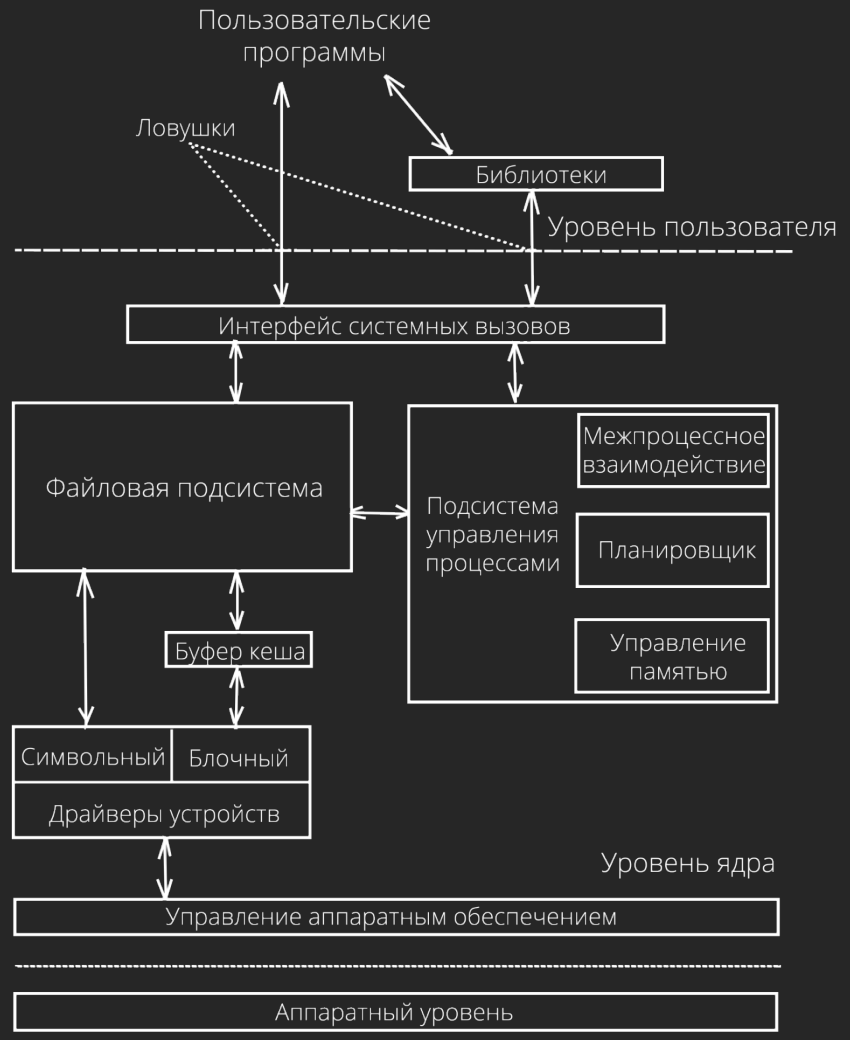
Минусы:

- Многоуровневые системы сложны для разработки.
- Менее эффективны, чем монолитные.

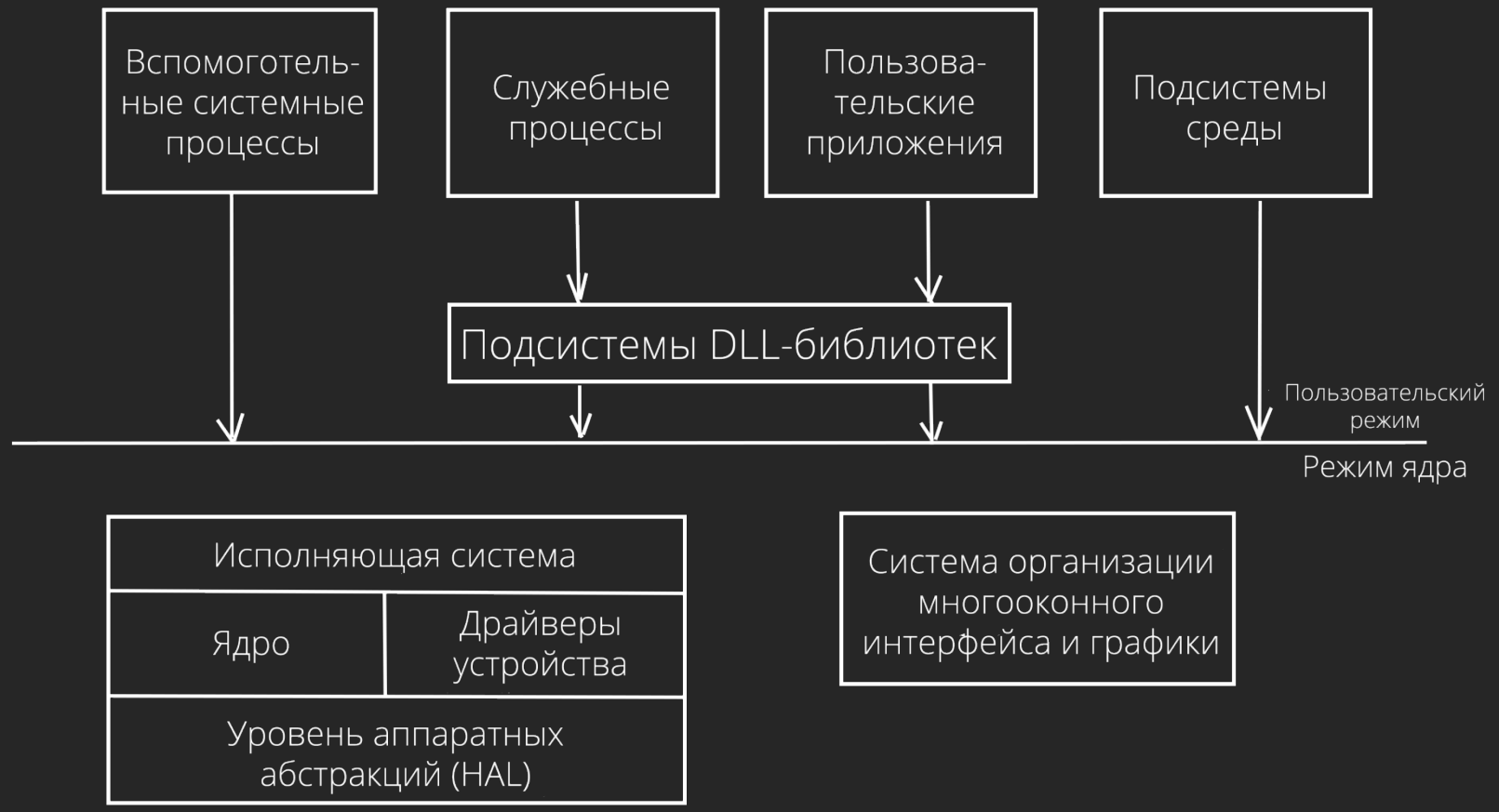
Традиционная архитектура UNIX

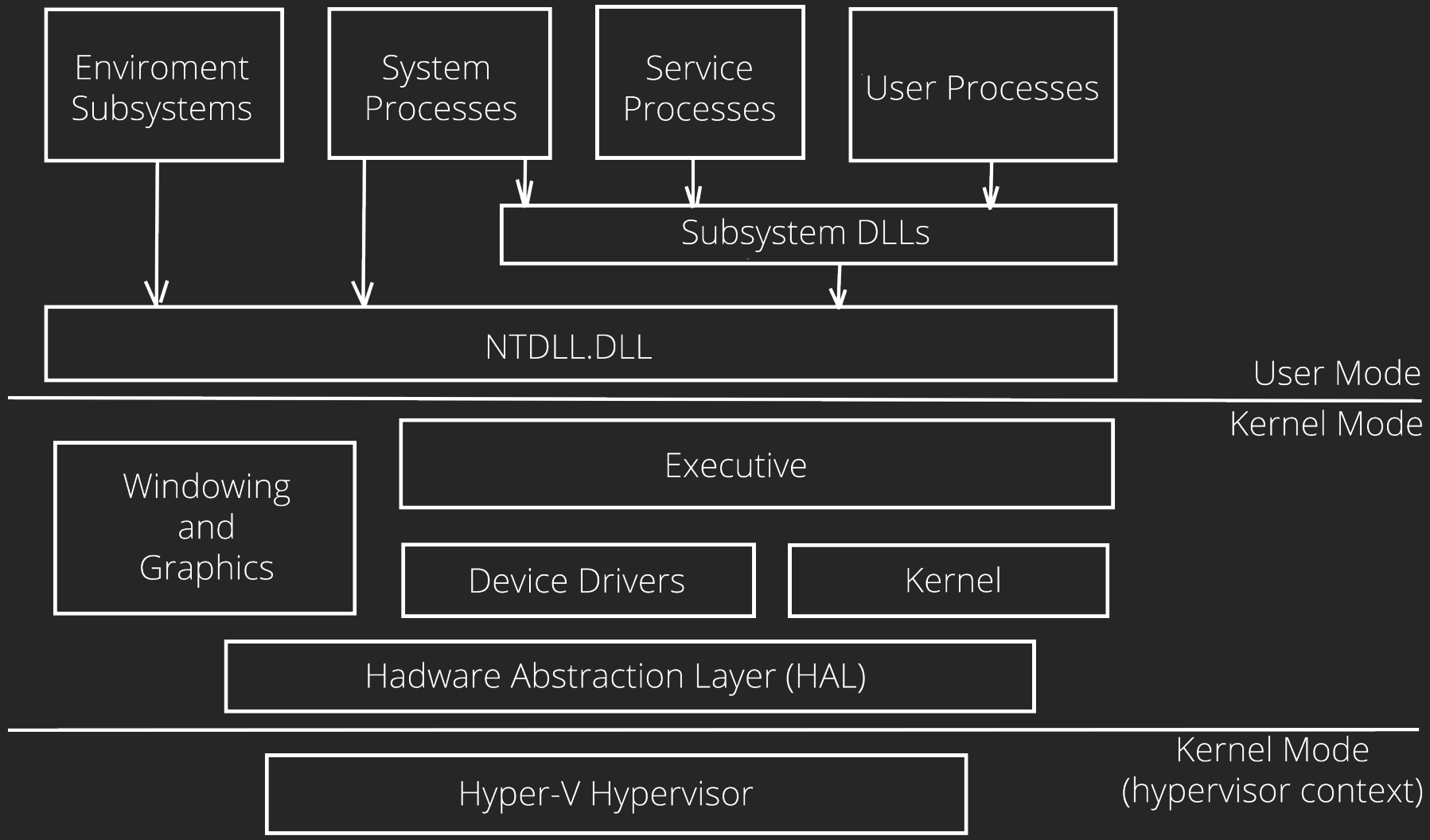


Традиционная архитектура UNIX

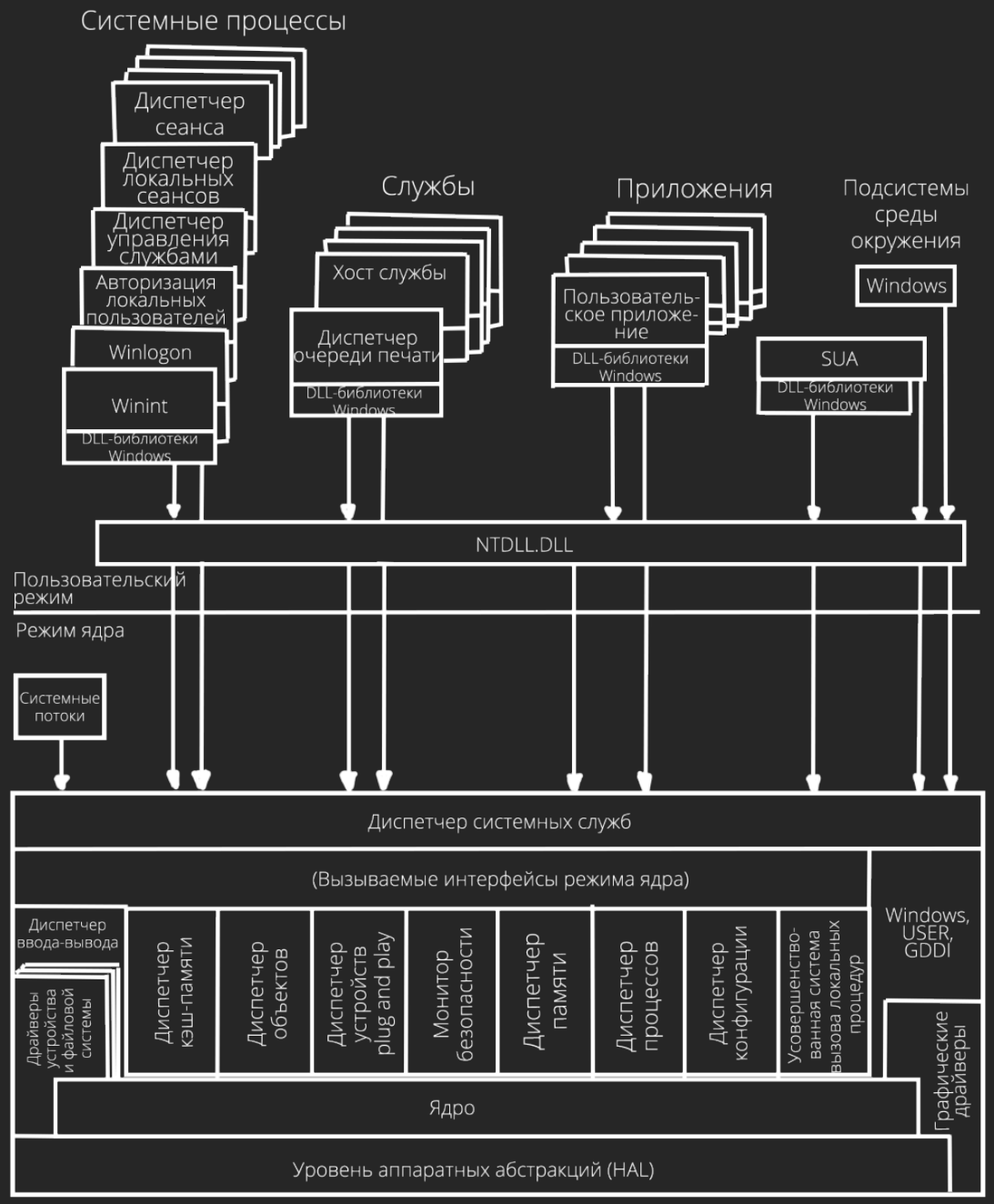


Упрощенное представление архитектуры Windows



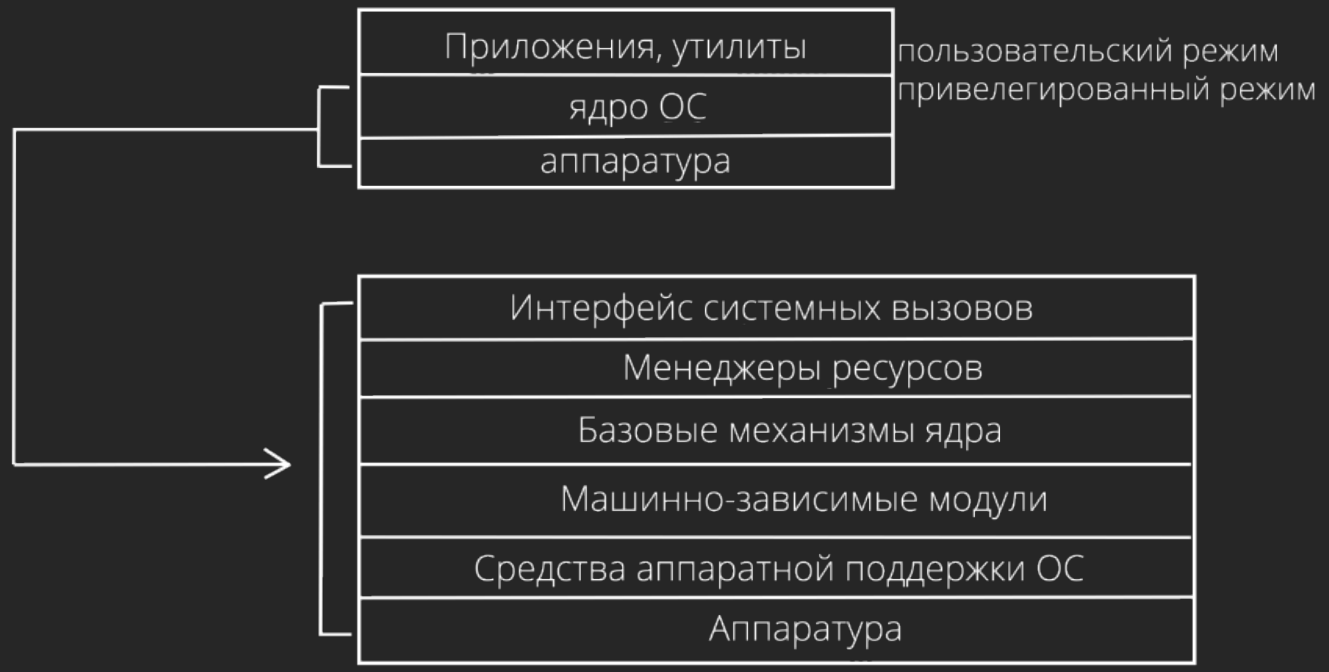

Enviroment Subsystems
System Processes
Service Processes
User Processes
Subsystem DLLs
NTDLL.DLL
User Mode
Kernel Mode
Windowing and Graphics
Executive
Device Drivers
Kernel
Hadware Abstraction Layer (HAL)
Kernel Mode (hypervisor context)
Hyper-V Hypervisor

Архитектура Windows



Аппаратные интерфейсы (шины, устройства ввода-вывода, прерывания, интервальные таймеры, DMA, управление кэш-памятью и т.д.)

Многослойная модель ядра



Ядро может состоять из следующих слоев:

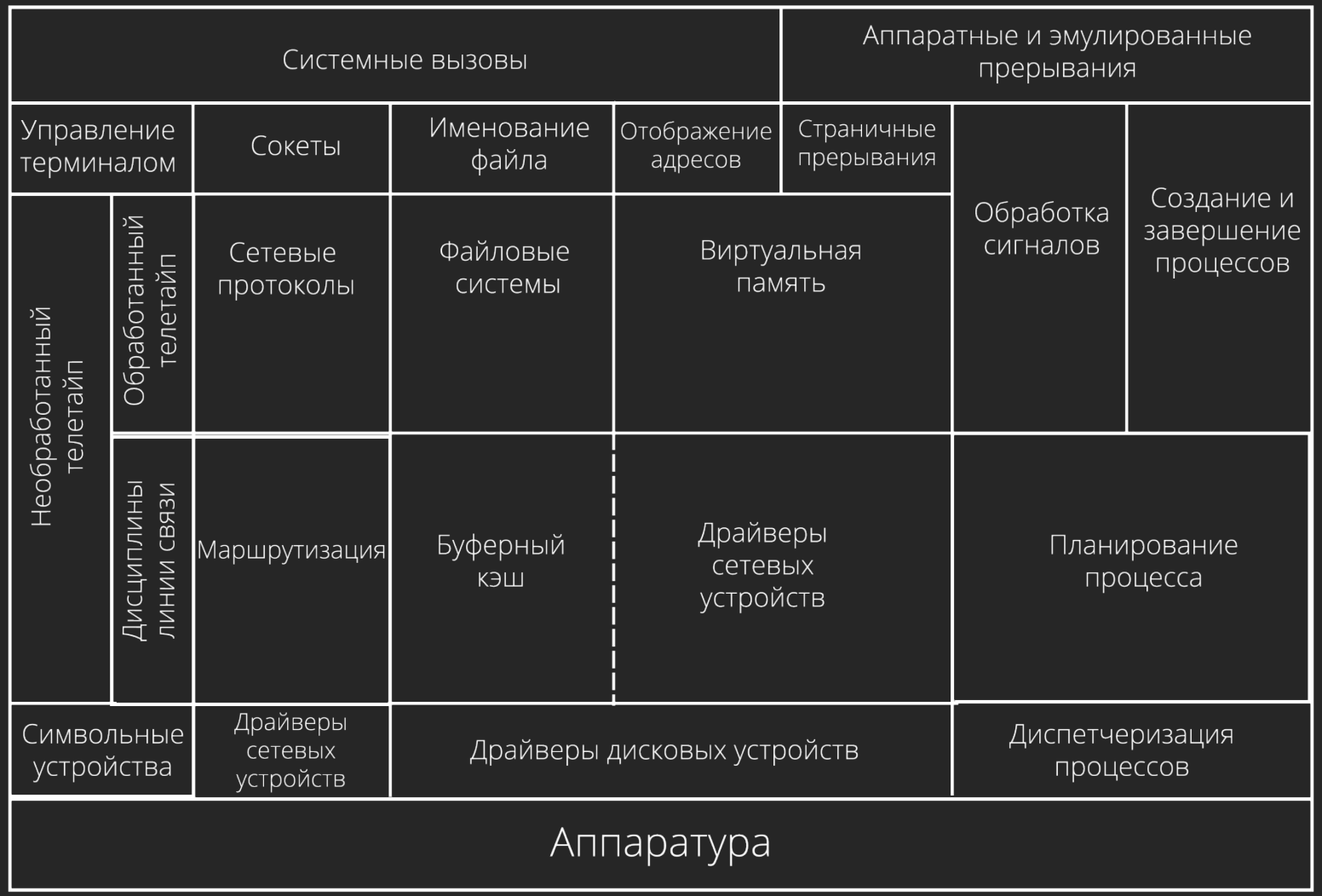
- машинно-зависимые компоненты ОС; в идеале этот слой полностью экранирует вышележащие слои ядра от особенностей аппаратуры (пример – слой HAL ОС Windows NT);

- базовые механизмы ядра, этот слой выполняет наиболее примитивные операции ядра, реализует решения о распределении ресурсов, принятые на более высоком уровне;

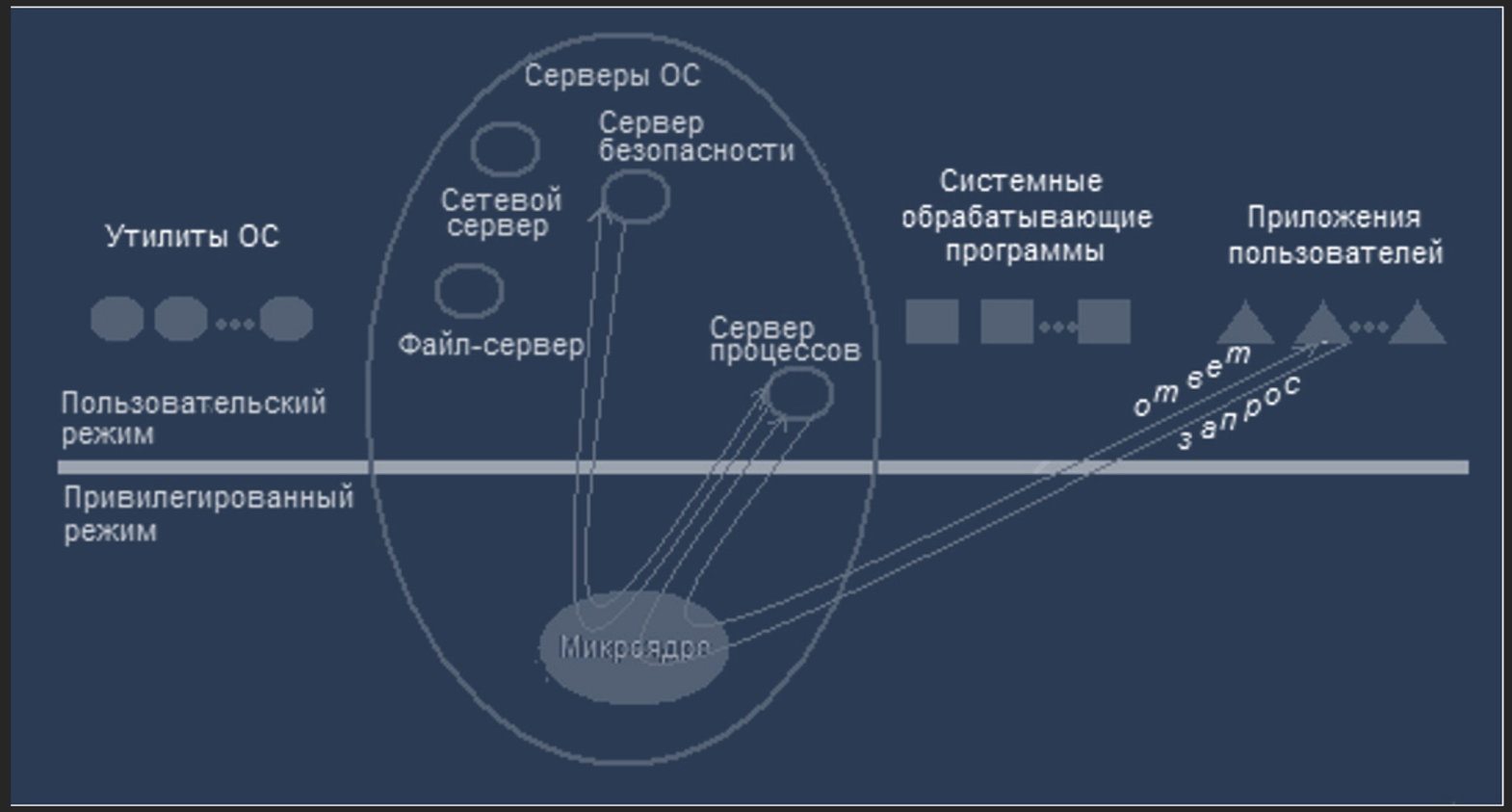
- менеджеры ресурсов; слой состоит из мощных функциональных модулей, реализующих стратегические задачи по управлению основными ресурсами ОС;

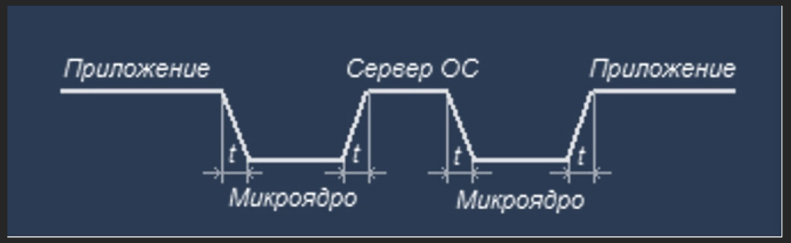
- интерфейс системных вызовов взаимодействует непосредственно с приложениями и системными утилитами, образуя прикладной программный интерфейс ОС.

Ядро ОС UNIX


Системные вызовы
Аппаратные и эмулированные прерывания
Управление терминалом
Сокеты
Именование файла
Отображение адресов
Страничные прерывания
Обработка сигналов
Создание и завершение процессов
Необработанный телетайп
Обработанный телетайп
Сетевые протоколы
Файловые системы
Виртуальная память
Дисциплины линии связи
Маршрутизация
Буферный кэш
Драйверы сетевых устройств
Планирование процесса
Символьные устройства
Драйверы сетевых устройств
Драйверы дисковых устройств
Диспетчеризация процессов
Аппаратура

Микроядерная архитектура





$t_{з\Sigma}=4t$

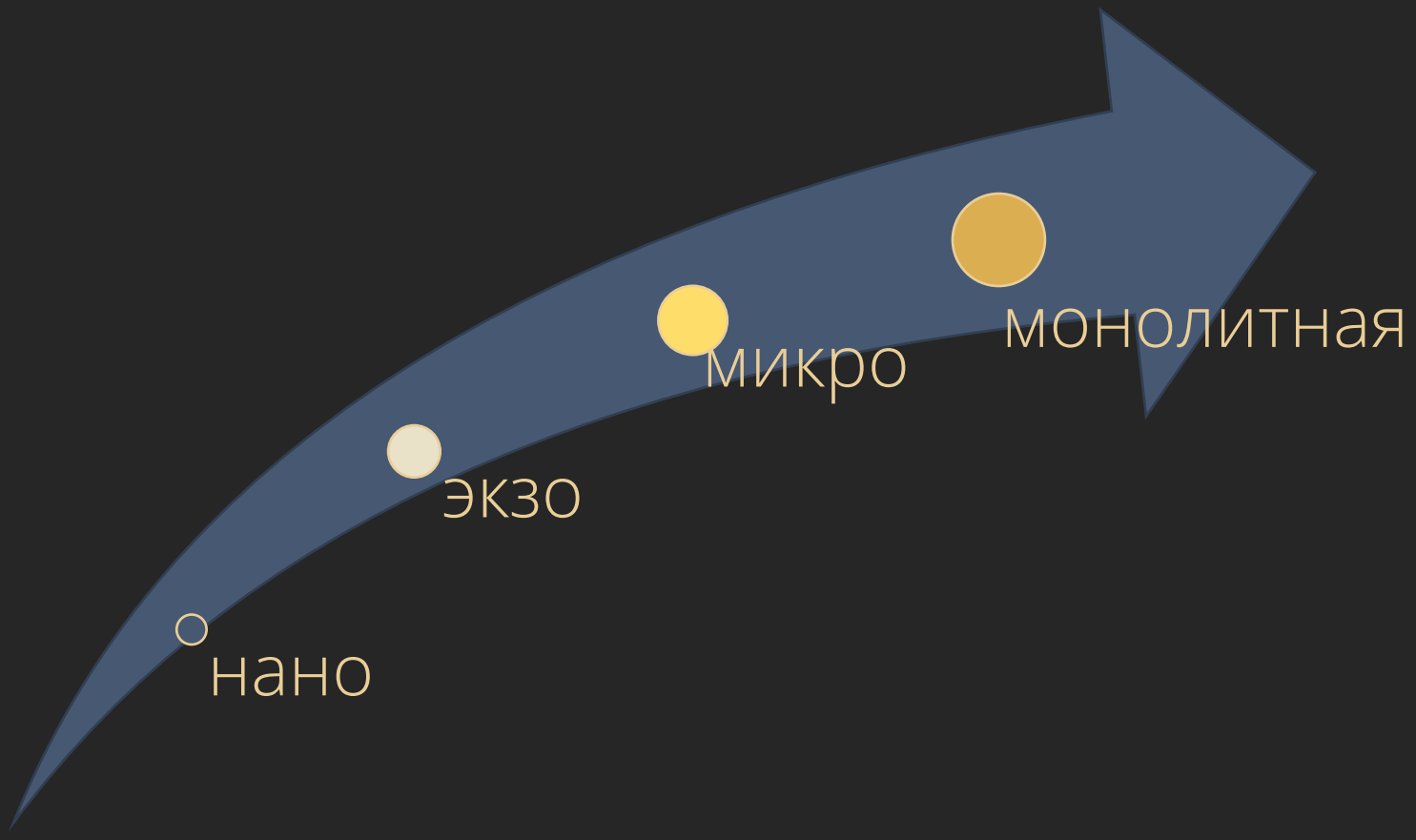
Экзоядро — ядро операционной системы компьютеров, предоставляющее лишь функции для взаимодействия между процессами и безопасного выделения и освобождения ресурсов.

Наноядро — архитектура ядра операционной системы компьютеров, в рамках которой упрощённое и минималистичное ядро выполняет лишь одну задачу — обработку аппаратных прерываний, генерируемых устройствами компьютера. После обработки прерываний от аппаратуры наноядро посылает информацию о результатах обработки вышележащему программному обеспечению при помощи того же механизма прерываний. Возможна минимальная поддержка потоков: создание и переключение.



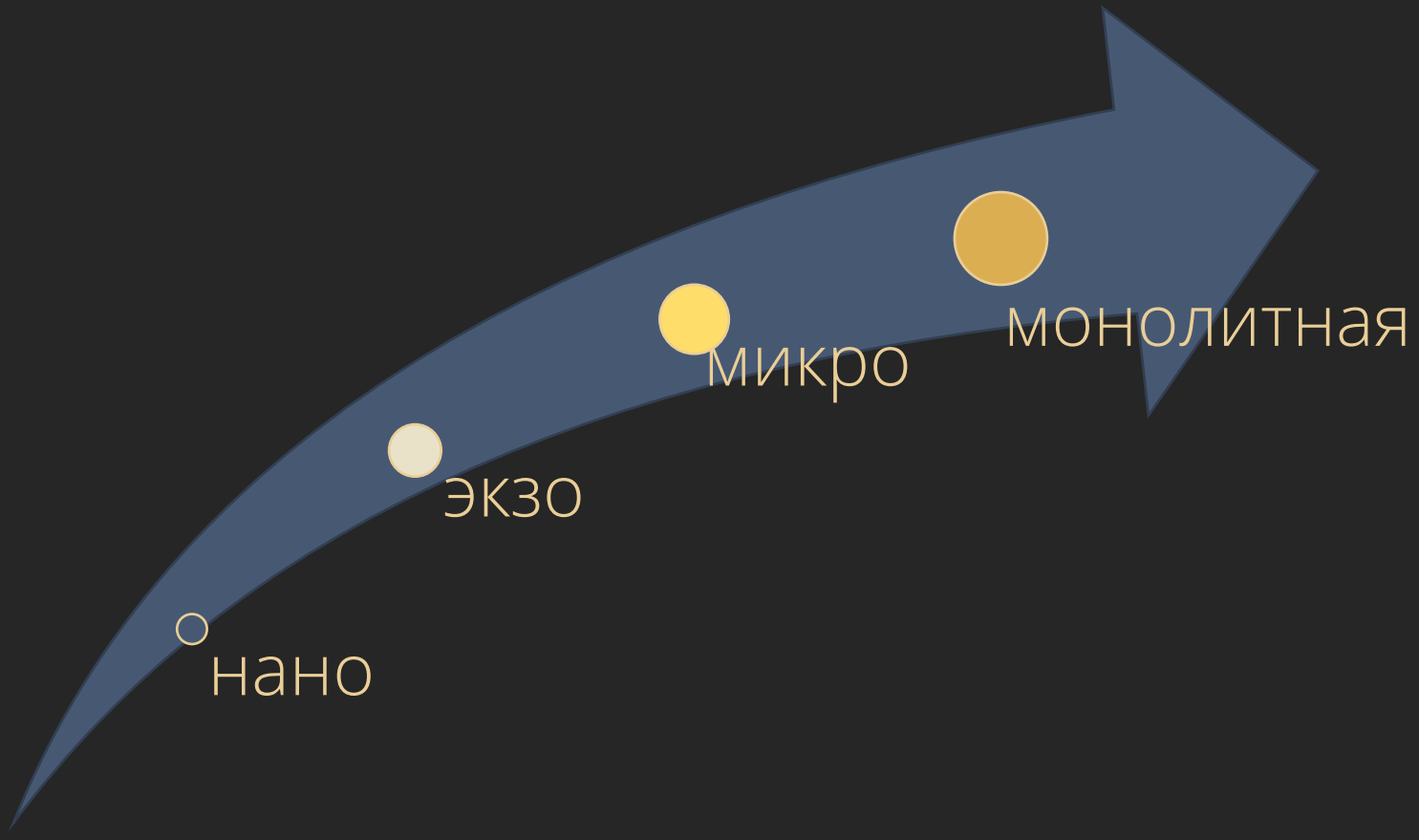
Сравнение моделей архитектур ОС

Производительность



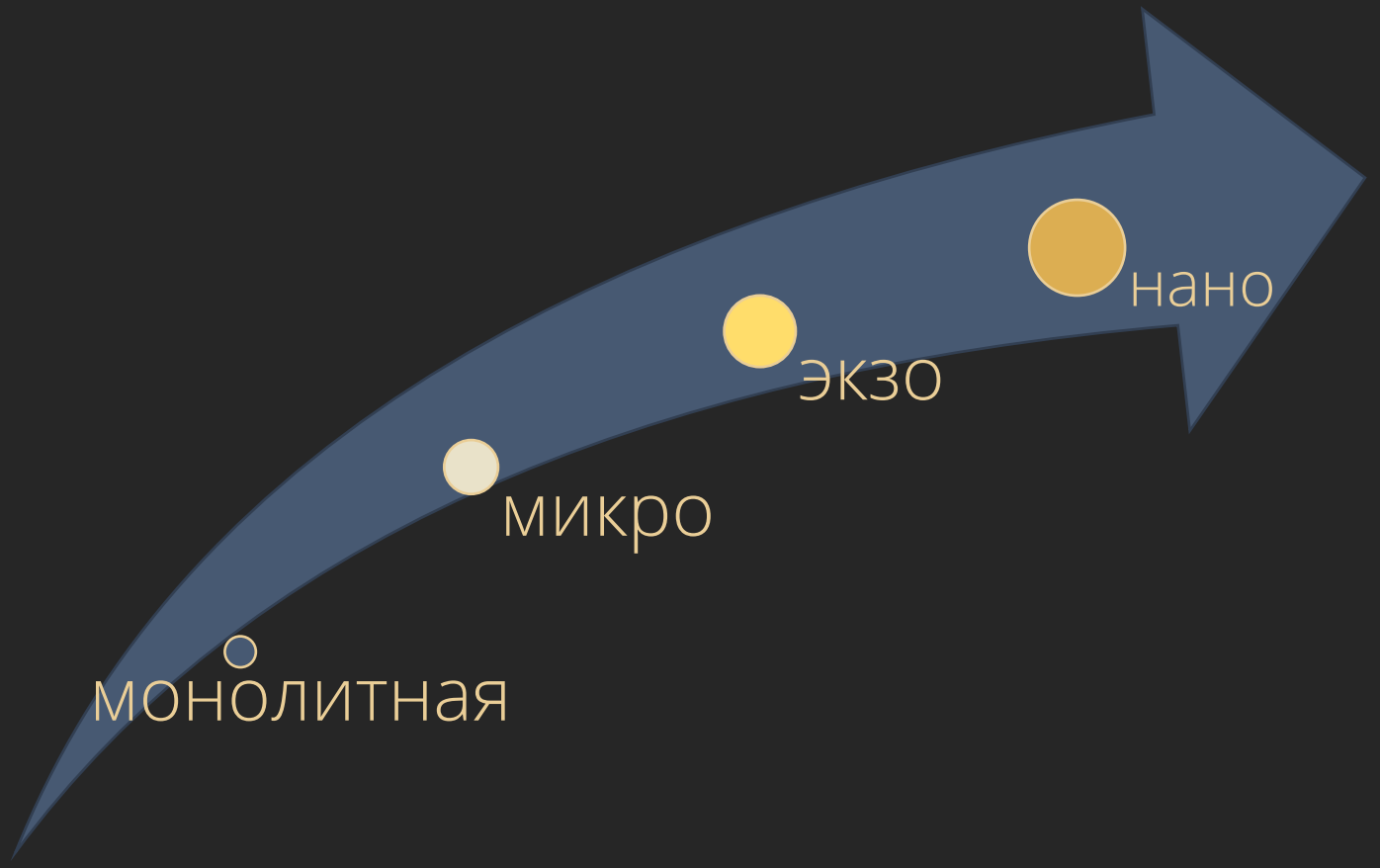
Сравнение моделей архитектур ОС

Объём привилегированного кода

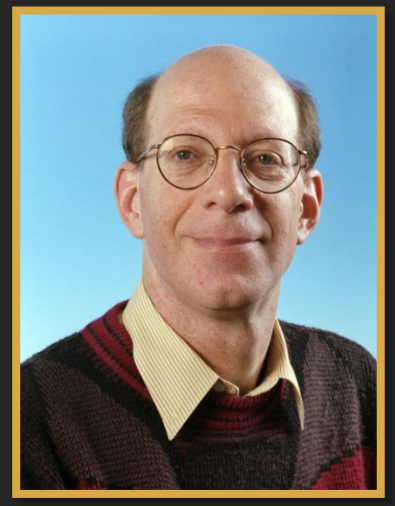


Сравнение моделей архитектур ОС

Защищенность ядра



Спор Эндрю Таненбаума и Линуса Торвальдса



Minix
Микроядро

VS



Linux
Монолитная архитектура

